



DATA CHALLENGE & PREREQUISITES

DAIR³

Jackson State U · U Michigan · UT San Antonio



DAIR³

The Data and AI Intensive Research with Rigor and Reproducibility (DAIR³) program is funded by Award 5R25GM151182 (2023-2028) of the National Institute of General Medical Sciences, one of the institutes of the National Institutes of Health of the United States. The principal investigators are Jing Liu (University of Michigan) and Juan B. Gutiérrez (University of Texas at San Antonio).

<https://dair-3.org/>

Faculty:

Clifton Addison, Associate Professor of Biostatistics, Jackson State University
Yalanda Barner, Assistant Professor of Health Policy and Management, Jackson State University
Johann Gagnon-Bartsch, Associate Professor of Statistics, University of Michigan
Juan B. Gutiérrez, Principal Investigator; Professor of Mathematics, University of Texas at San Antonio
Gregory Hunt, Assistant Professor of Mathematics, College of William and Mary
Brenda Jenkins, Director of Training and Education, Jackson State University
Erin Kaleba, Director, Data Office for Clinical and Translational Research, University of Michigan
Jing Liu, Principal Investigator; MIDAS Executive Director, University of Michigan
Jodyn Platt, Associate Professor of Learning Health Sciences; Associate Professor of Health Management and Policy, University of Michigan Medical School
Suraj Rampure, Lecturer III in Electrical Engineering and Computer Science, University of Michigan
Kerby Shedden, Professor of Statistics, University of Michigan

Research Associates:

Taofeeq Oluwatosin Togunwa, PhD Student, Department of Learning Health Sciences, University of Michigan
Simranjeet Singh, Master's Student, School of Information, University of Michigan

Administrative Support:

Kelly Psilidis, Faculty Training Program Manager, University of Michigan
Michele Randolph, Evaluation Specialist, Marsal School of Education, University of Michigan
Annick Rougée, Evaluation Associate II, Marsal School of Education, University of Michigan
Johanna Massé, Evaluation Associate II, Marsal School of Education, University of Michigan

Each faculty member develop their own module.

The final document was compiled and edited by Juan B. Gutiérrez, Ph.D. Professor of Mathematics, juan.gutierrez3@utsa.edu

The University of Texas at San Antonio

May 22, 2026

Contents

Introduction	7
1 Environment Setup	11
1.1 Required: Install Python	11
1.2 Optional (but <u>strongly</u> encouraged): Install a L ^A T _E X Distribution . .	13
1.3 Required: Install Visual Studio Code (VSC)	14
1.4 Required: Visual Studio Code extensions	14
1.5 Optional: Windows Terminal	16
1.6 Required: Install Git	16
1.7 Required: Open Git Bash (or any terminal)	16
1.8 Required: Open the DAIR3 project in Visual Studio Code	17
1.9 Required: Install Python libraries	17
1.10 Optional: Jupyter Notebook	17
1.11 Optional: Python commands in Terminal	18
1.12 Troubleshoot	18
1.13 Optional: C/C++ compilers	19
1.14 Optional: Installing PostgreSQL on Linux	19
1.14.1 Install Required Utilities	19
1.14.2 Add the PostgreSQL Repository	19
1.14.3 Import the Repository Signing Key	19
1.14.4 Install PostgreSQL	20
1.14.5 Verify the Installation	20
1.14.6 Start the Database Server	20
1.14.7 Access the PostgreSQL Shell	20
2 Command-Line Interfaces, Virtual Environments, and Shell Scripting	21
2.1 What Is a File?	21
2.1.1 Files and text	21
2.1.2 The filesystem: folders and paths	21
2.1.3 The working directory	22
2.2 Introduction: What Is a Command-Line Interface?	22
2.2.1 A Concrete Analogy	22
2.2.2 Why Developers and Researchers Use the CLI	22
2.2.3 The Two Major CLI Worlds	23
2.2.4 Your First Commands	23

2.3	Two Worlds: Windows and Linux/Mac	24
2.3.1	Paths and Separators	24
2.3.2	Scripts and Executability	24
2.3.3	Environment Variables and PATH	24
2.4	WSL: A Linux Environment Inside Windows	24
2.4.1	What WSL Gives You	25
2.4.2	Installing WSL	25
2.4.3	The Elegant Solution: Visual Studio Code and the WSL Extension	25
3	Introduction to Programming Environments	27
3.1	Before the editor: writing your first program in a text file	27
3.2	Introduction	28
3.2.1	Compiled Languages	29
3.2.2	Scripting Languages	30
3.2.3	Comparative Examples	30
3.2.4	What is Python?	31
3.2.5	Python Installation	32
3.2.6	Executing Python Commands	32
3.2.7	The VSC Python Editor: Executing A Simple Program . . .	34
3.2.8	Executing Functions	37
3.3	Introduction to NumPy	37
3.3.1	Matrix Operations	39
4	Python Virtual Environments and Workflow Automation	42
4.1	Traditional Package Management: <code>pip</code>	42
4.1.0.1	Windows (CMD or PowerShell)	42
4.1.0.2	Linux / WSL	43
4.1.1	Activating a Virtual Environment	43
4.1.1.1	Windows	43
4.1.1.2	Linux / WSL	43
4.1.2	Installing Packages	43
4.1.3	Deactivating	44
4.1.4	Installing the DAIR3 Required Libraries	44
4.2	Modern Package Management: <code>uv</code>	45
4.2.1	Why <code>uv</code> Matters Today	45
4.2.2	What <code>uv</code> Offers	45
4.2.3	Installing <code>uv</code>	46
4.2.3.1	Windows (PowerShell)	46
4.2.3.2	Linux / WSL	46
4.2.4	Creating and Managing Environments with <code>uv</code>	46
4.2.4.1	Creating an Environment	46
4.2.4.2	Activating an Environment	47
4.2.4.3	Installing Packages	47
4.2.4.4	Declaring Dependencies with <code>pyproject.toml</code> . . .	47
4.2.5	<code>requirements.txt</code> vs. <code>uv.lock</code>	48

4.3	Level 1: Script Files	48
4.3.1	The Windows Solution: Batch Files	48
4.3.1.1	ActivateEnv.bat	48
4.3.1.2	DAIR3.bat	49
4.3.2	The Linux Solution: Shell Scripts	50
4.3.2.1	Understanding Shell Script Syntax	50
4.3.2.2	ActivateEnv.sh	50
4.3.2.3	DAIR3.sh	51
4.3.3	Making a Script Executable	51
4.3.4	Adding Your Scripts to PATH	52
4.3.5	The uv Approach: <code>ActivateUVEnv</code>	52
4.3.5.1	ActivateUVEnv.bat (Windows)	52
4.3.5.2	ActivateUVEnv: Linux / WSL Shell Function	53
4.4	The Critical Difference: Executing vs. Sourcing	55
4.4.1	Child Shells	55
4.4.2	Sourcing: Running in the Current Shell	55
4.4.3	The Windows Parallel	56
4.5	Level 2: Shell Functions, The Elegant Linux Approach	56
4.5.1	What Is a Shell Function?	56
4.5.2	Defining the <code>ActivateEnv</code> Function	56
4.5.3	Defining the <code>DAIR3</code> Function	57
4.5.4	Activating the New Functions	57
4.5.5	Why Windows Has No Clean Equivalent	58
4.6	Putting It All Together	58
4.6.1	The Full WSL Experience	58
4.7	Quick Reference Cheat Sheet	59
4.8	Summary	60
5	Numerical Operations	62
5.1	Bits, Bytes, and Floating Point	62
5.1.1	The IEEE Standard for Floating-Point Arithmetic	63
5.1.2	A Bestiary of Floating Point Anomalies	64
5.2	Introduction to NumPy	67
5.2.1	Matrix Operations	68
5.3	Specific tasks for this lab	70
6	Introduction to $\LaTeX$	71
6.1	Your First $\LaTeX$ Document	71
6.2	A More Complete Example	72
6.3	Document Structure	77
6.4	Mathematics in $\LaTeX$: A Cheat Sheet	77
6.4.1	Common Mathematical Notation	78
6.5	Figures	80
6.6	Tables	81
6.7	Bibliography	81
6.8	LaTeX Workshop in Visual Studio Code	81

6.9	Summary	82
7	Version Control with Git and GitHub	83
7.1	Configuring SSH Authentication for GitHub on Linux	83
7.1.1	Generate an SSH Key	83
7.1.2	Start the SSH Agent	83
7.1.3	Add the Public Key to GitHub	83
7.1.4	Verify the Connection	84
7.1.5	Clone a Repository Using SSH	84
7.2	Configuring SSH Authentication for GitHub on Windows	84
7.2.1	Opening Git Bash	84
7.2.2	Generating an SSH Key	85
7.2.3	Starting the SSH Agent	85
7.2.4	Adding the Public Key to GitHub	86
7.2.5	Verifying the Connection	86
7.2.6	Cloning a Repository Using SSH	87
7.2.7	Pushing to Multiple Remote Repositories	88
7.2.7.1	Concepts	88
7.2.7.2	Listing Registered Remotes	88
7.2.7.3	Adding a Second Remote	88
7.2.7.4	Pushing to Each Remote	89
7.2.7.5	Checking Whether a Remote Exists Before Pushing	89
7.2.7.6	Managing Remotes	90
7.2.7.7	Alternative: a Single Remote with Multiple Push URLs	90
7.3	Summary	91
8	Local LLM Infrastructure with Ollama	92
8.1	Introduction	92
8.2	Hardware Inventory	92
8.2.1	Dell Precision 7770 (Application Host)	92
8.2.2	NVIDIA Spark (LLM Server)	93
8.3	Network Topology	93
8.3.1	Switch vs. Router	94
8.3.2	MAC Address Filtering	94
8.3.3	Identifying Network Interfaces	94
8.4	Installing Ollama	95
8.4.1	Linux (Spark)	95
8.4.2	Windows (Dell)	95
8.4.3	Conda Environments	95
8.5	Configuring Ollama for Network Access	95
8.6	Memory Budget and Context Window Management	96
8.6.1	The Governing Formula	96
8.6.2	The Modelfile Pattern for Large Models	97
8.6.2.1	Linux (Spark and WSL)	97
8.6.2.2	Windows (PowerShell)	97

8.6.3	Hardware Agent Capacity	98
8.6.4	Thinking Models and Token Budgets	98
8.7	Configuring API Keys	99
8.8	The FOO Framework and Agent Feasibility	100
8.8.1	Minimum Agent Count	100
8.8.2	Agent Feasibility Score	100
8.8.3	Reference Configurations	100
8.9	Single-Machine Configuration (Dell)	101
8.9.1	Environment Variable	101
8.9.2	Model Selection for Single-Machine Development	102
8.10	Two-Machine Configuration	102
8.10.1	Architecture	102
8.10.2	Environment Variables	103
8.10.3	Connectivity Verification	103
8.11	Capacity Testing	103
8.11.1	Platform Considerations	104
8.11.2	Ollama CLI Reference	104
8.12	Deployment Parity	105
8.13	Setting Up Ollama with a GUI on Ubuntu	105
8.13.1	Step 1: Install Ollama Backend	105
8.13.2	Step 2: Deploy Open WebUI via Docker	105
8.13.3	Step 3: Access the Interface	106
8.13.4	Technical Considerations	106
9	2026 Data Challenge - Vital Statistics	107
9.1	Problem Description	107
9.2	Data Description	107
9.3	Identifying Underweight Cutoff Points	108
9.4	Data Files	110
9.5	Criteria	110
10	Analysis Activities	112
10.1	A single year of vital statistics	112

Introduction

This document is a companion reference for participants in the DAIR3 program. It covers the software environment, programming foundations, and data challenge materials needed before and during the summer bootcamp. It is *not* intended to be read cover to cover. Think of it as a reference manual: use the routing table on the next page to identify which chapters apply to your background, complete those chapters before the first session, and return to the others as needed during the program.

This document is version-locked to the 2026 cohort and is planned for obsolescence on December 31, 2026. After that date, retrieve the current edition from the program repository at <https://github.com/DAIR3/DAIR3-Workshop>. The companion curriculum document, *DAIR3 2026 Curriculum*, covers the seven program units delivered during the bootcamp; this document covers the prerequisites and data challenge only.

Information

You are **not** expected to read this document from cover to cover. Identify the chapters relevant to your background using the routing table on the next page, complete those before the first session, and return to the remaining chapters only as the need arises during the program. Most participants will skip at least two or three chapters entirely.

Conventions Used in This Document

Inline Q&A prompts. Throughout the chapters, red prompts marked **Record your explanation as answer #k.** appear at key points. These are classroom discussion prompts that do not require a written response here; use the fillable form in Chapter ?? to record responses to the curated subset of questions selected for the progress record.

Callout boxes. Four types of callout box appear in the text:

- **Information** (teal border): reference material and neutral context.
- **Tip** (blue border): practical guidance and shortcuts.
- **Warning** (magenta border): critical cautions that, if ignored, will cause errors or data loss.

- **Note** (orange border): items requiring attention but not critical.

Code listings. All commands intended to be typed in a terminal are shown in monospace code blocks. Windows commands use a dark background; Linux/macOS commands use a lighter background. Commands that work identically on all platforms use the neutral listing style.

Cross-references. Blue underlined text is a clickable hyperlink. References to chapters, sections, figures, and tables are also clickable in the PDF.

Chapter Dependencies

Figure 1 shows the dependency structure of the chapters. A solid arrow indicates a hard dependency: the target chapter requires the source to be completed first. A dashed arrow indicates recommended order only.

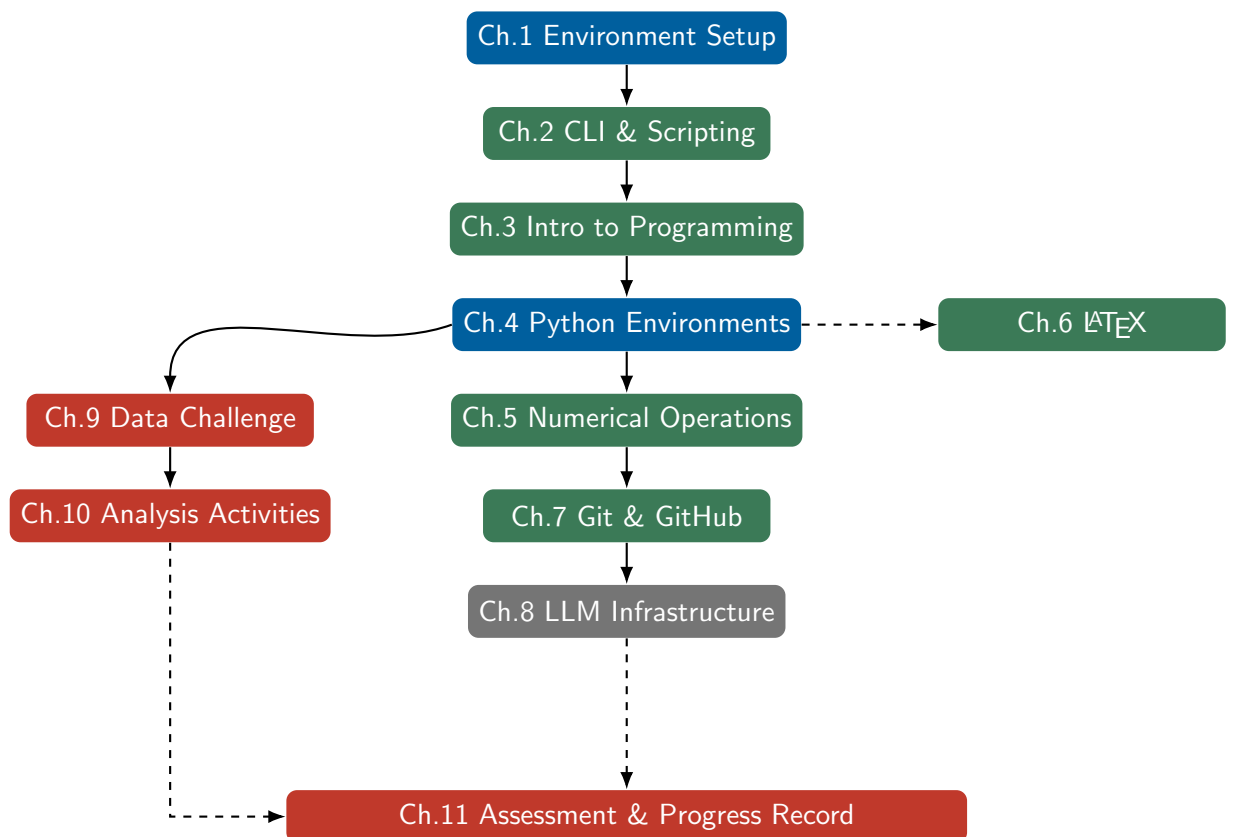


Figure 1: Chapter dependency structure. Solid arrows indicate hard dependencies; dashed arrows indicate recommended order. Node color encodes category: **Required**, **Essential**, **Desirable**, **Optional**.

Which Chapters Do You Need?

Use Table 1 to identify your reading path based on your prior experience. Chapters marked **Required** must be completed; **Skim** means read the section headings and run the exercises but skip the explanatory prose; **Skip** means the chapter is optional for you.

Table 1: Recommended reading path by prior experience.

Profile	<i>Env. Setup</i>	<i>CLI</i>	<i>Intro Prog.</i>	<i>Python Envs.</i>	<i>Numerical Ops.</i>	<i>L^AT_EX</i>	<i>Git</i>	<i>LLM</i>	<i>Data Challenge</i>	<i>Analysis</i>
New to programming	•	•	•	•	•	•	•	S*	•	•
Know Python, new to Linux/CLI	•	•	S	•	S	•	•	S*	•	•
Experienced programmer	•	S	–	•	–	•	S	S*	•	•

• = Required S = Skim – = Skip *Chapter 8: all profiles complete Section 8.7 only; infrastructure sections are optional.

Progress Record

The final chapter, Chapter ?? (Assessment and Progress Record), is a fillable PDF form. It organizes the key completion items from all chapters into four categories: Required, Essential, Desirable, and Optional. Open it in Adobe Acrobat Reader to fill and save your responses. Refer to it throughout the program as a checklist for readiness.

Estimated Completion Times

Table 2 shows estimated completion times per chapter. Times assume no prior exposure to the topic; students with background in a given area will complete it faster.

Table 2: Estimated completion time per chapter.

Chapter	Title	Estimated Time
1	Environment Setup	2 hours
2	Command-Line Interfaces and Shell Scripting	2 hours
3	Introduction to Programming Environments	2 hours
4	Python Virtual Environments and Workflow Automation	1.5 hours
5	Numerical Operations with NumPy	2 hours
6	Introduction to $\LaTeX$	1.5 hours
7	Version Control with Git and GitHub	1.5 hours
8	Local LLM Infrastructure with Ollama	1 hour (students) / 3 hours (instructors)
9	Data Challenge	1 hour (reading)
10	Analysis Activities	4–6 hours
11	Assessment and Progress Record	Ongoing
Total (student path)		$\approx$ 19–21 hours



Chapter 1

Environment Setup

This chapter walks through the installation of all software required for the DAIR3 project. Work through the required sections in order before the first session. Optional sections can be completed at any time. The instructions are written primarily for Windows but include equivalent steps for macOS and Linux throughout.

Familiarity with the command line and with file paths is helpful but not required here. These concepts are covered fully in Chapter 2, which you can read after completing this chapter.

1.1 Required: Install Python

Install any recent Python version from <https://www.python.org/downloads/>. Go to your downloads and double click on the install file to start installation. By default the Python installer for Windows places its executables in the user's App-Data directory, so that it doesn't require administrative permissions. This works for most scenarios. If you're the only user on the system, you might want to place Python in a higher-level directory (e.g. `C:\Python` or `/usr/local/bin`) to have a shorter path to the binaries (sometimes you will need that). Depending on your preferences, either select "*Install Now*" or "*Customized installation*" (my preference). Please make sure you select "*Add python.exe to path*"; this will save you a few headaches later.

In the window "Optional Features" select all features

Select "*Install Python X.XX for all users*" if you can and want. This requires administrative privileges. Select the folder of your choice for the install location.

For PC: If you did not add `python.exe` to the PATH during installation, wait until the installation is complete. Then open File Explorer and right-click on "This PC." Select "Properties" from the context menu. In the System window that opens, click on "Advanced system settings" in the left-hand sidebar. In the new window, ensure the "Advanced" tab is selected and click the "Environment Variables" button near the bottom. Under the "System variables" section, find and double-click on the variable named "Path." This will open a window where you can add the path to the Python executable manually.

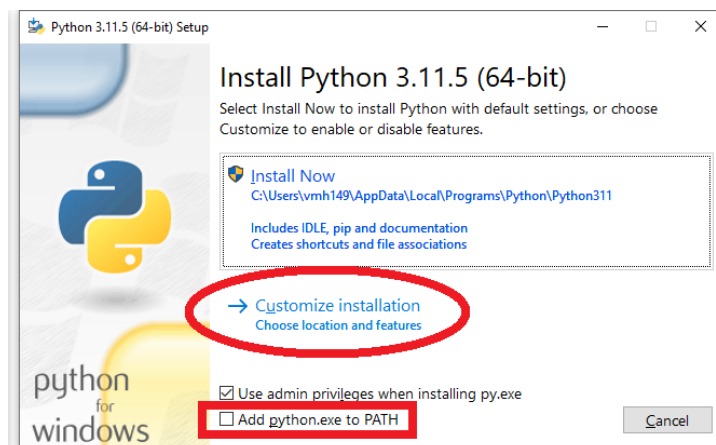


Figure 1: Python installer showing the “Add python.exe to PATH” checkbox at the bottom.

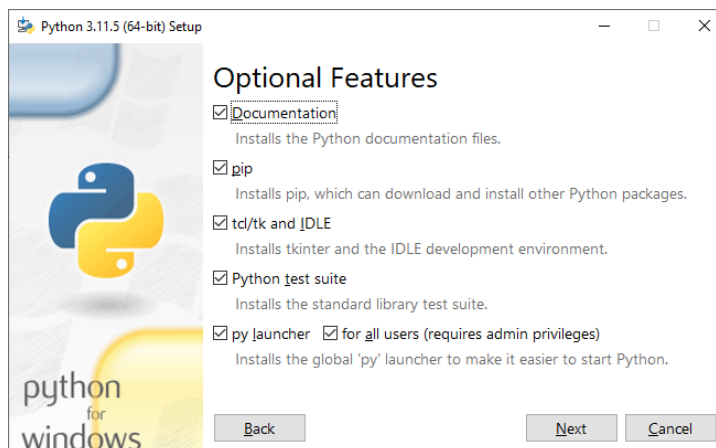


Figure 2: Python Optional Features screen: select all features.

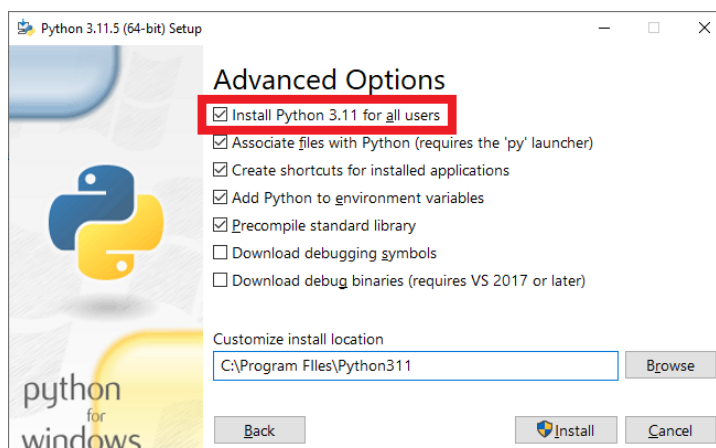


Figure 3: Python Advanced Options screen showing “Install for all users” selected.

For Mac: If you did not ensure Python was added to your system's PATH during installation, you can manually do so using the following steps on a Mac. After installation is complete, open the Terminal application. Enter the command `which python3` or `which python` to find the path to the installed Python binary. Copy that path. Then open your shell configuration file in a text editor. This will typically be `~/.zshrc` if you are using the Zsh shell (default on newer versions of macOS) or `~/.bash_profile` if you are using Bash. Add the line `export PATH="/path/to/python:$PATH"`, replacing `/path/to/python` with the path you copied. Save the file and close the editor. In the Terminal, run `source ~/.zshrc` or `source ~/.bash_profile` depending on the file you edited, so the new PATH is loaded into your environment.

1.2 Optional (but strongly encouraged): Install a L^AT_EX Distribution

L^AT_EX is a high-quality typesetting system designed for the creation of technical and scientific documents. Rather than formatting text visually, authors write plain-text source files using a markup language that describes the structure and meaning of the content (such as sections, equations, and references). These source files are then compiled into professionally formatted documents (typically PDFs). L^AT_EX excels at typesetting equations, managing cross-references, and producing consistent, publication-quality output, making it the standard tool for many fields in science, engineering, and mathematics.

Information

L^AT_EX is particularly well suited for interaction with Large Language Models

L^AT_EX is particularly well suited for interaction with Large Language Models (LLMs) because it is a purely textual, declarative markup language that closely resembles structured source code. Unlike binary document formats (e.g., Word or PDF), a L^AT_EX document exposes its full logical structure—sections, equations, environments, and references—in plain text. This makes it directly interpretable, editable, and generatable by LLMs, which are fundamentally trained on and optimized for text and code. As a result, LLMs can reliably assist with tasks such as drafting mathematical content, refactoring structure, verifying notation, and even debugging compilation issues. In this sense, L^AT_EX serves as a natural interface between human mathematical reasoning and machine-assisted generation, enabling precise, reproducible, and automatable document workflows.

L^AT_EX is required if you intend to compile `.tex` documents locally. The program's documents are maintained as L^AT_EX source files in the repository. For authoring and compiling your own documents, install the distribution for your platform.

Verify the installation on any platform with:

```
pdflatex --version
```

Table 1.1: Recommended $\LaTeX$ distributions by platform

Platform	Distribution	Download
Windows	MiKTeX	https://miktex.org
macOS	MacTeX	https://www.tug.org/mactex/
Linux (Ubuntu/Debian)	TeX Live	<code>sudo apt install texlive-latex-extra texlive-fonts-recommended latexmk</code>

Usage of $\LaTeX$ for document preparation, including a mathematics notation cheat sheet, is covered in Chapter 6.

1.3 Required: Install Visual Studio Code (VSC)

If you are new to Python, install Visual Studio Code (VSC): There are many options to write Python code, and you could have a favorite one. I must pick one to deliver instruction, and experience has shown me that VSC offers the least amount of friction.

In VSC, there are two main options: “*User Installer*” and “*System Installer*”. Use the User Installer if you want to install only for the current user. For all users, use System installer; I prefer this choice because it installs VSC in `C:\Program Files\Microsoft VS Code`.

Visual Studio Code is available for Windows, macOS, and Linux; an identical open-source alternative called VSCodium is available at <https://vscodium.com>.

1.4 Required: Visual Studio Code extensions

Select the Extensions icon on the left and install the following Visual Studio Code libraries

1. **Required:** Python extension for Visual Studio Code.
2. **Optional:** Also install Jupyter (this also installs Pylance, Jupyter Keymap, Jupyter Notebook Renderers, Jupyter SlideShow). Pay attention to the publisher of the extension; use the ones by Microsoft.
3. **Optional:** Install C/C++ for Visual Studio Code, C/C++ Themes & C/C++ Extension Pack.
4. **Optional:** Install the extension “**LaTeX Workshop**” by James Yu. This extension provides syntax highlighting, live preview, and one-click build. Its use is described in Section 6.8.

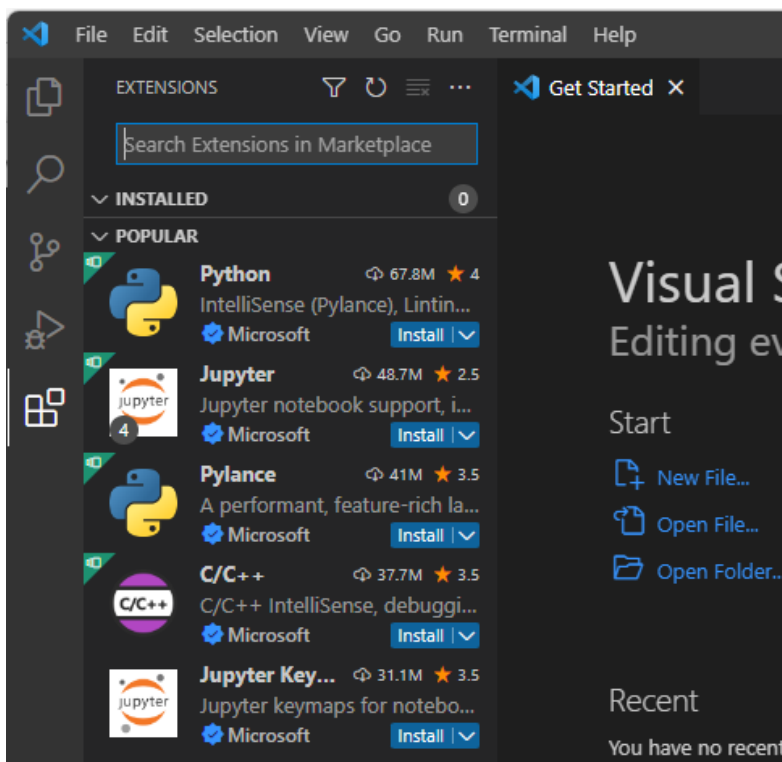


Figure 4: VS Code Extensions marketplace showing the Python and Jupyter extensions by Microsoft.

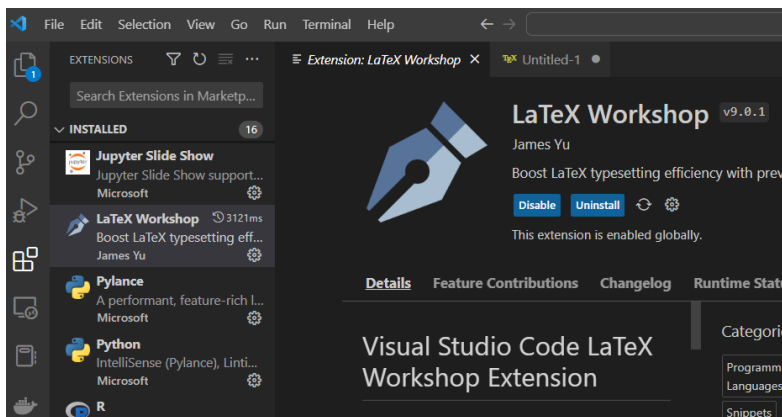


Figure 5: LaTeX Workshop extension page in VS Code.

1.5 Optional: Windows Terminal

For Windows users, install Windows Terminal. This Microsoft app allows you to use a multi document window with tabs for different command consoles like PowerShell, Ubuntu, DOS, Git, etc. You will need this if you want to complete the steps related to Linux in the next section of this document.

1.6 Required: Install Git

Install Git for Windows. For Mac, install Git for Mac. Linux also has a version available. In the second screen, select Visual Studio Code as your Git editor. Use all other default options.

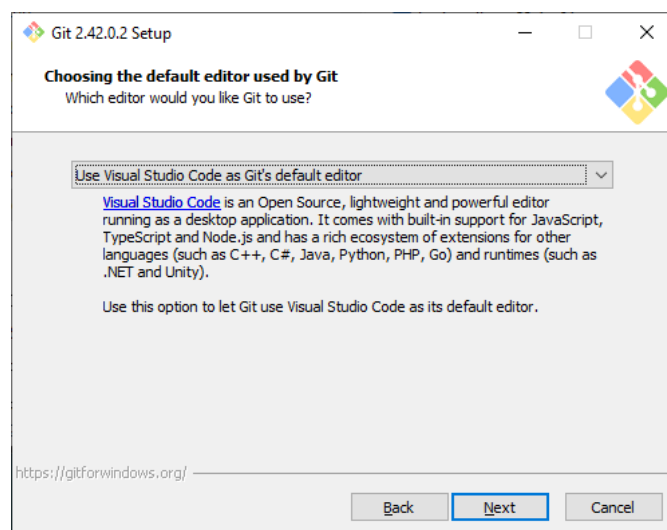


Figure 6: Git setup screen showing Visual Studio Code as the default editor.

1.7 Required: Open Git Bash (or any terminal)

The command line allows you to control your computer by typing instructions, providing precise access to files and system features. For example, typing `ls` on a Unix-like system such as Linux or macOS lists the files and directories in the current location, such as `ls /home/user/Documents` to display the contents of the Documents folder. On Windows, the `dir` command serves the same function, so `dir C:\Users\Alice\Desktop` shows what is on the Desktop. To move between directories, the `cd` command is used; `cd Downloads` changes the working directory to Downloads, while `cd ..` moves one level up. These commands allow efficient navigation and inspection of the file system through the terminal.

Now that Git is installed, clone the DAIR3 repository. Open Git Bash (Windows) or a terminal (macOS/Linux), navigate to the folder where you want to store the project, and run:

```
git clone https://github.com/DAIR3/DAIR3-Workshop.git
```

This creates a folder called **DAIR3-Workshop** containing all workshop materials. You only need to do this once. To update your local copy later, navigate into the folder and run `git pull`.

1.8 Required: Open the DAIR3 project in Visual Studio Code

To open the DAIR3 project in VSC, open a terminal, navigate into the **DAIR3-Workshop** folder you cloned in the previous section, and run:

```
cd DAIR3-Workshop
code .
```

The period after `code` instructs VSC to use the current folder as the working folder. All workshop files will be visible in the VSC file explorer.

1.9 Required: Install Python libraries

Python libraries for the data challenge are installed inside a virtual environment. Complete Chapter 4 to set up your environment, then install the required libraries as described there.

1.10 Optional: Jupyter Notebook

Optional: Now, create a Jupyter Notebook.

1. Abundant relevant information regarding Jupyter in VSC is available at <https://code.visualstudio.com/docs/datascience/jupyter-notebooks>
2. Activate the environment you want to use (see Chapter 4).
3. You need to attach the kernel of your virtual environment. For example, if I have a virtual environment called `venv`:

```
python -m ipykernel install --user --name=venv
```

4. Open VS Code in your project folder (see Section 1.8).
5. In VSC, run the command **Create:New Jupyter Notebook** from the Command Palette (Ctrl+Shift+P) or by creating a new `.ipynb` file in your workspace.
6. You might receive a Security Alert regarding the firewall. Allow access.
7. After you enter a command, you might see the following message:
Install it.



Figure 7: Windows Defender Firewall alert: allow access for Visual Studio Code.

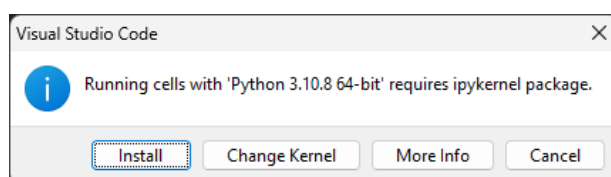


Figure 8: VS Code prompt to install the ipykernel package: click Install.

1.11 Optional: Python commands in Terminal

Finally, you can simply enter Python commands in the Terminal. Open a Command Prompt and type the following command:

```
python
```

This will allow you to enter python commands, e.g. `1+1`

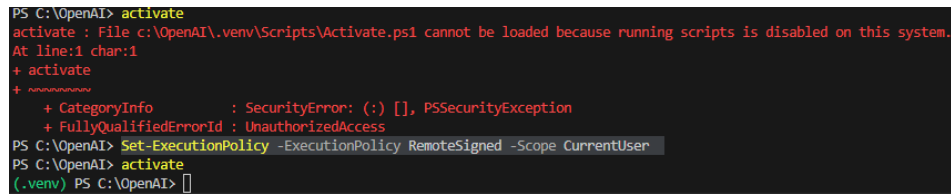
1.12 Troubleshoot

The following errors have been reported by previous participants. If you encounter a different issue, consult the DAIR3 repository issue tracker at <https://github.com/DAIR3/DAIR3-Workshop/issues>.

As reported by Lorena Roa de La Cruz, you could see an error when trying to activate a virtual environment. This is due to configuration of permissions. If this happens, run the command:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

After that, run the script `activate`.



```

PS C:\OpenAI> activate
activate : File c:\OpenAI\.venv\Scripts\Activate.ps1 cannot be loaded because running scripts is disabled on this system.
At line:1 char:1
+ activate
+ ~~~~~
+ CategoryInfo          : SecurityError: (:) [], PSSecurityException
+ FullyQualifiedErrorId : UnauthorizedAccess
PS C:\OpenAI> Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
PS C:\OpenAI> activate
(.venv) PS C:\OpenAI>

```

Figure 9: PowerShell execution policy error and fix: running `Set-ExecutionPolicy` followed by `activate`.

1.13 Optional: C/C++ compilers

If you'd like to code in C or C++, you have several options, including GCC, the GNU Compiler Collection, but it requires some effort to install the prerequisites. From Microsoft there is Visual Studio Community Edition (VSCE), which encapsulates the complexity of installing multiple compilers (including the .NET framework SDK). I recommend you install VSCE with all “Workloads” in “Web & Cloud” and “Desktop & Mobile” (except Python).

1.14 Optional: Installing PostgreSQL on Linux

Many Linux distributions ship with older versions of PostgreSQL in their default repositories. To install the latest stable version, it is recommended to use the official PostgreSQL package repository maintained by the PostgreSQL Global Development Group.

The following procedure installs the most recent stable release on Ubuntu or Debian systems.

1.14.1 Install Required Utilities

First install utilities required to retrieve and verify packages.

```

1 sudo apt update
2 sudo apt install curl ca-certificates

```

1.14.2 Add the PostgreSQL Repository

Add the official PostgreSQL repository to the system package sources.

```

1 sudo sh -c 'echo "deb http://apt.postgresql.org/pub/repos/apt \
2 $(lsb_release -cs)-pgdg main" \
3 > /etc/apt/sources.list.d/pgdg.list'

```

1.14.3 Import the Repository Signing Key

Import the PostgreSQL repository signing key.

```

1 curl -fsSL https://www.postgresql.org/media/keys/ACCC4CF8.asc \
2 | sudo gpg --dearmor -o /usr/share/keyrings/postgresql.gpg

```

Update the repository configuration to reference the key.

```
1 sudo nano /etc/apt/sources.list.d/pgdg.list
```

Modify the line to read in a single line (the character `\` means line return; it is here just to indicate that all is one line; delete it and join the following three lines of text into a single line):

```
1 deb [signed-by=/usr/share/keyrings/postgresql.gpg] \
2 http://apt.postgresql.org/pub/repos/apt \
3 $(lsb_release -cs)-pgdg main
```

1.14.4 Install PostgreSQL

Update the package index and install the latest PostgreSQL version.

```
1 sudo apt update
2 sudo apt install postgresql-16
```

1.14.5 Verify the Installation

Confirm that PostgreSQL was installed successfully.

```
1 psql --version
```

The output should indicate the installed PostgreSQL version.

1.14.6 Start the Database Server

If the service is not already running, start the PostgreSQL cluster.

```
1 sudo systemctl start postgresql
```

Verify the server status:

```
1 sudo systemctl status postgresql
```

1.14.7 Access the PostgreSQL Shell

To open the PostgreSQL interactive terminal:

```
1 sudo -u postgres psql
```

Exit the shell using:

```
1 \q
```

Chapter 2

Command-Line Interfaces, Virtual Environments, and Shell Scripting

2.1 What Is a File?

2.1.1 Files and text

A file is a named sequence of bytes stored on a storage device. From the computer's perspective, all files are sequences of bytes. What distinguishes a Python script from a photograph is not the storage mechanism but the interpretation: a text editor interprets bytes as characters; an image viewer interprets them as pixel values. A Python file is plain text: every character you type is stored as a byte, and the Python interpreter reads those bytes as instructions. This is why a Word document cannot serve as a Python program; Word stores invisible formatting bytes alongside the text.

2.1.2 The filesystem: folders and paths

Every file lives inside a folder (also called a directory). Folders can contain other folders, forming a tree. The location of any file can be described by a *path*: a sequence of folder names separated by a delimiter, ending in the filename.

Table 2.1: Path syntax on each operating system.

Operating tem	Sys-	Example path
Windows		C:\Users\Alice\Documents\hello.py
macOS		/Users/alice/Documents/hello.py
Linux		/home/alice/Documents/hello.py

Windows uses backslashes and a drive letter; macOS and Linux use forward slashes and start from a root /.

2.1.3 The working directory

At any moment, the CLI is “inside” a particular folder, called the *current working directory* (CWD). Commands that refer to files without a full path are interpreted relative to that folder. When you type `python hello.py`, the interpreter looks for `hello.py` in the CWD. If the file is elsewhere, the command fails. Understanding the CWD is the single most common source of confusion for new CLI users and the root cause of most “file not found” errors.

The CLI provides the commands to navigate the filesystem, inspect its contents, and execute files; these are introduced in the next section.

“The command line is the most powerful interface ever invented. It is also the most intimidating. This chapter aims to change that.”

2.2 Introduction: What Is a Command-Line Interface?

When you use a computer, you almost certainly interact with it through a **Graphical User Interface** (GUI): windows, icons, menus, and buttons that you click with a mouse. GUIs are intuitive and visually rich, but they have a fundamental limitation: every action you can take must first be anticipated and implemented as a button or menu item by the software developer. If you need to do something the GUI was not designed for, you are stuck.

A **Command-Line Interface** (CLI) solves this by letting you communicate with the computer in *text*. You type a command; the computer executes it and responds in text. This is not a step backward; it is a different, and in many contexts more powerful, mode of interaction.

2.2.1 A Concrete Analogy

Think of the difference between a restaurant with a fixed menu and a restaurant where you can describe exactly what you want to the chef. A GUI is the fixed menu: fast, friendly, and good enough for most customers. A CLI is talking directly to the chef: it requires more knowledge of ingredients, but the results can be precisely what you need, and you can combine them in ways the menu never anticipated.

2.2.2 Why Developers and Researchers Use the CLI

- **Automation.** A sequence of commands can be saved in a file (called a *script*) and replayed perfectly every time.
- **Precision.** Every option and parameter is explicit and readable.
- **Remote access.** Servers in the cloud rarely have a graphical desktop; the CLI is often the only way in.

- **Speed.** Renaming 10,000 files takes one command; doing it by hand in a GUI might take hours.
- **Reproducibility.** You can share a script with a colleague and know they will get exactly the same result.

2.2.3 The Two Major CLI Worlds

This chapter focuses on two CLI environments:

Windows Command Prompt / PowerShell The native CLI tools in Microsoft Windows. Batch files (`.bat`) and PowerShell scripts (`.ps1`) automate tasks here.

Bash (Linux / macOS) The dominant CLI on Linux systems and macOS. Shell scripts (`.sh`) automate tasks here.

These two worlds use different syntax, different path conventions (backslash \ vs. forward slash /), and different scripting philosophies. A key goal of this chapter is to help you move fluidly between them.

2.2.4 Your First Commands

The best way to demystify the CLI is to try it. Table 2.2 shows equivalent commands in both environments.

Table 2.2: Equivalent CLI commands: Windows vs. Linux/macOS

Task	Windows (CMD)	Linux / macOS (Bash)
Print current directory	<code>cd</code>	<code>pwd</code>
List files	<code>dir</code>	<code>ls -la</code>
Change directory	<code>cd foldername</code>	<code>cd foldername</code>
Go up one level	<code>cd ..</code>	<code>cd ..</code>
Create a folder	<code>mkdir myfolder</code>	<code>mkdir myfolder</code>
Delete a file	<code>del file.txt</code>	<code>rm file.txt</code>
Copy a file	<code>copy a.txt b.txt</code>	<code>cp a.txt b.txt</code>
Print text to screen	<code>echo Hello</code>	<code>echo Hello</code>
Clear the screen	<code>cls</code>	<code>clear</code>

Tip

In both environments, pressing the **Up arrow** key recalls your previous commands, and pressing **Tab** auto-completes file and directory names. These two habits alone will dramatically speed up your work at the CLI.

2.3 Two Worlds: Windows and Linux/Mac

Before writing any scripts, it helps to understand the philosophical differences between how Windows and Linux handle the command line. These differences are not arbitrary; they reflect decades of divergent design history.

2.3.1 Paths and Separators

Windows uses *backslashes* and drive letters to describe file locations:

```
1 C:\Users\DrG\Documents\DAIR3\main.py
```

Linux uses *forward slashes* and a single unified file tree rooted at `/`:

```
1 /home/drg/DAIR3/main.py
```

2.3.2 Scripts and Executability

On Windows, the file extension determines whether a file is executable. Files ending in `.bat`, `.cmd`, or `.exe` are treated as programs.

On Linux, the extension is irrelevant. Executability is a *permission* stored on the file itself. A file must be explicitly granted execute permission before the system will run it. This is done with the `chmod` command, which we cover in Section 4.3.3.

2.3.3 Environment Variables and PATH

Both systems use an environment variable called `PATH` to find programs. When you type a command, the shell searches each directory listed in `PATH` in order and runs the first match it finds.

- **Windows:** `PATH` is managed through System Properties → Environment Variables, or via the `setx` command.
- **Linux:** `PATH` is set in configuration files such as `~/.bashrc` (for the Bash shell) or `~/.zshrc` (for Zsh), which are read every time a new terminal session starts.

2.4 WSL: A Linux Environment Inside Windows

For Windows users who want the power of Linux scripting without leaving their familiar environment, Microsoft provides the **Windows Subsystem for Linux** (WSL). WSL is not an emulator or a virtual machine in the traditional sense; it is a compatibility layer built into Windows that runs a real Linux kernel alongside the Windows kernel.

2.4.1 What WSL Gives You

- A full Bash shell (or Zsh, Fish, etc.)
- A complete Ubuntu (or other distribution) file system
- Access to thousands of Linux command-line tools via `apt`
- The ability to read and write Windows files from Linux (your Windows `C:` drive appears at `/mnt/c/`)
- GPU access for machine learning workloads

2.4.2 Installing WSL

Open PowerShell or CMD *as Administrator* and run:

```
1 wsl --install
```

Listing 2.1: Installing WSL from PowerShell

This installs WSL 2 and the Ubuntu distribution by default. After a restart, a Ubuntu terminal will open and ask you to create a username and password. That account is independent of your Windows login.

Note

Throughout the remainder of this chapter, all Linux instructions apply equally to WSL on Windows and to a native Linux machine. Treat them as the same environment.

2.4.3 The Elegant Solution: Visual Studio Code and the WSL Extension

One of the most powerful aspects of WSL is its integration with **Visual Studio Code** (VS Code), Microsoft's free code editor. Installing the **WSL extension** in VS Code creates a seamless bridge between your Windows desktop and your Linux environment:

- You open VS Code from within the WSL terminal by typing `code .`
- VS Code runs its interface on Windows but executes all code, terminal commands, extensions, and debuggers *inside Linux*
- The file explorer shows your Linux file system
- The integrated terminal is a Bash shell
- Python environments, linters, and formatters all resolve to their Linux versions

This means you never have to choose between the Linux scripting ecosystem and the Windows desktop experience: you get both simultaneously. From this point forward, when we say “open VS Code,” we mean launching it from the WSL terminal with `code` .

Tip

To install the WSL extension, open VS Code, press **Ctrl+Shift+X** to open the Extensions panel, and search for “WSL” (publisher: Microsoft). One click installs it.

Chapter 3

Introduction to Programming Environments

3.1 Before the editor: writing your first program in a text file

A Python program is plain text stored in a file with a `.py` extension. Any tool that saves plain text (not formatted text) can create one. Before using a sophisticated editor, you will create and run a program using the simplest text editor available on your system. This establishes a fundamental idea: the editor is irrelevant; the file is what matters.

Table 3.1: Opening a plain text editor on each operating system.

Operating System	How to open the editor	Critical step before typing
Windows	Press <code>Win + R</code> , type <code>notepad</code> , press <code>Enter</code>	In the <code>Save As</code> dialog, set “Save as type” to <code>All Files (*.*)</code> before saving; otherwise the file saves as <code>hello.py.txt</code>
macOS	Open Spotlight (<code>Cmd + Space</code>), type <code>TextEdit</code> , press <code>Enter</code>	Go to Format > Make Plain Text before typing anything; <code>TextEdit</code> defaults to rich text, which corrupts code
Linux	Press <code>Ctrl + Alt + T</code> to open a terminal, then type <code>gedit &</code> and press <code>Enter</code>	No additional step required; <code>gedit</code> saves plain text by default

Exercise P-1: Your first Python program

Type the program below exactly as shown into your text editor. Pay attention to the indentation: the spaces before `print(n)` and `if n == 5:` are not decorative; they are required Python syntax and the program will fail without them. Save the file as `hello.py` in a folder you can find (your Desktop is a good choice). Apply the critical step in Table 3.1 for your operating system before saving.

Word processors are not text editors

Microsoft Word, Google Docs, and LibreOffice Writer save files with invisible formatting markup. A Python file saved from one of these tools will fail to run. Use only the editors listed in Table 3.1 or VSC.

```

1 # Comments begin with the character #
2 # Comments are not part fo the program.
3 # They can be used to introduce visual markers for clarity
4 # -----
5 # This program exemplifies inoput-output (I/O),
6 # loops, conditionals, and assignment
7 print("This is programming!")
8 k = 10
9 for n in range(k):
10     print(n)
11     if n == 5:
12         print("here")

```

The `#` line is a comment; Python ignores it. Comments exist for humans. `k = 10` is an assignment; it creates a variable named `k` and stores the value 10. `for n in range(k):` is a loop; everything indented below it executes 10 times. `if n == 5:` is a conditional; the indented block below only runs when the condition is true.

The program exists as a file, but it does nothing until it is executed by the Python interpreter. Now that you have completed Chapter 2, you have all the tools needed to run it. Open a terminal, navigate to the folder where you saved `hello.py`, and run:

```
python hello.py
```

On macOS and Linux, you may need to use `python3` instead of `python`. The output should match what you would expect from reading the code.

3.2 Introduction

There is a large number of programming languages. For the purpose of scientific computing, we will distinguish two main families of languages: compiled vs. scripting languages. They represent two different paradigms of programming language

design, each with its own advantages and trade-offs. The key difference lies in when and how the source code you write is translated into machine code that the computer can understand and execute.

In the realm of scientific computing, the choice between using a compiled language like C++ and a scripting language like Python often comes down to the specifics of the project at hand, including its complexity, required execution speed, and the need for direct hardware access.

3.2.1 Compiled Languages

Compiled languages are those where the source code is entirely translated into machine code before execution. This translation is done by a compiler, a program that takes the entire source code as an input and generates an executable file. Examples of compiled languages include C++, Java, Rust, Go, and Swift.

Compiled languages offer several important benefits including:

- **Performance:** Since the code is precompiled into machine code, compiled programs usually run faster and more efficiently than interpreted ones.
- **Type checking:** Many compiled languages have strict type systems, which catch many type-related errors at compile time, before the code is run.
- **Security:** The source code is not distributed with the software, which makes it harder for others to reverse engineer the software.

However, compiled languages also have some drawbacks:

- **Development speed:** The edit-compile-run cycle can be time consuming, particularly for large software projects.
- **Portability:** The generated executable is usually specific to a type of processor and operating system. Availability for different operating systems or processors might require specific source code alterations and independent compilations.

Compiled languages like C++ are translated into machine code before execution, making them very fast because they're executed directly by the computer's hardware. This can be a significant advantage when dealing with huge datasets or when computation speed is a critical factor.

However, the syntax of languages like C++ is more complex and it may require more lines of code to achieve the same result as a scripting language. Moreover, C++ lacks many of the built-in data analysis functionalities found in languages such as Python, MATLAB or R, so it's necessary to use external libraries like Eigen or Armadillo for matrix operations and other complex computations.

3.2.2 Scripting Languages

Scripting languages, on the other hand, are typically interpreted from text into machine instructions, line-by-line of code just in time for execution. While this can make scripting languages slower than compiled languages, they're often easier to write and read because of their high-level syntax. Examples of scripting languages include Python, MATLAB, R, JavaScript, Ruby, and PHP.

Scripting languages offer several advantages:

- **Ease of use:** Scripting languages are usually easier to learn and use. They have less verbose syntax and more built-in functionality.
- **Flexibility:** Since they are interpreted line-by-line, it's easier to make changes to your code and see their effect immediately without having to recompile.
- **Portability:** The same script can be run on any system that has the correct interpreter installed, without needing to change the code.

Among the drawbacks of scripting languages there are issues related to:

- **Performance:** Interpreted languages are generally slower than compiled languages because of the overhead of interpreting code line-by-line during execution.
- **Dependency on the interpreter:** The code can only be run on systems that have the correct interpreter installed.

Python is particularly well-suited for biomedical data science because it has numerous powerful libraries for data analysis (like Pandas, NumPy, and SciPy) and visualization (like Matplotlib and Seaborn). In addition, Python's simple, straightforward syntax makes it an excellent choice for quick prototyping and exploratory data analysis.

It must be noted that even with all the benefits of Python, the simplified MATLAB syntax for matrix operations facilitates algorithm development since the source code closely resembles mathematical notation.

3.2.3 Comparative Examples

Let us consider a simple example where we want to multiply two matrices A and B. The language examples are listed below in alphabetical order by the name of the language.

In C++:

```
1 #include <Eigen/Dense>
2 #include <iostream>
3
4 int main() {
5     Eigen::Matrix2f A;
6     A << 1, 2,
7         3, 4;
```

```

8   Eigen::Matrix2f B;
9   B << 5, 6,
10      7, 8;
11   std::cout << "The product AB is:\n" << A * B;
12 }

```

../source/IntroProgMatMul.c

In MATLAB:

```

1 A = [1, 2; 3, 4];
2 B = [5, 6; 7, 8];
3 disp('The product AB is:')
4 disp(A * B)

```

../source/IntroProgMatMul.m

In R:

```

1 A <- matrix(c(1, 3, 2, 4), nrow=2, ncol=2, byrow = TRUE)
2 B <- matrix(c(5, 7, 6, 8), nrow=2, ncol=2, byrow = TRUE)
3
4 print("The product AB is:")
5 print(A %*% B)

```

../source/IntroProgMatMul.r

In Python:

```

1 import numpy as np
2
3 A = np.array([[1, 2], [3, 4]])
4 B = np.array([[5, 6], [7, 8]])
5 print("The product AB is:\n", np.matmul(A, B))

```

../source/IntroProgMatMul.py

From the above example, it's clear that MATLAB code is much more concise and easier to understand than C++. However, if we were dealing with very large matrices or needed to perform this operation many times in a loop, the C++ code might run faster due to being a compiled language.

3.2.4 What is Python?

Python is an *interpreted high-level* programming language for general-purpose computing.

- *Interpreted* means that a program in Python has to be decoded line by line by another program called the Python interpreter and translated into something a computer can understand.
- *High-level* means that there is a strong abstraction of the elements of the hardware of the computer; for instance, memory does not need to be pre-allocated in Python to create a matrix, whereas the C language (a low-level language) requires it. You can think of the level of the language as how close the instructions are to the metal of the machine.

- The process of language interpretation is called *compilation*, that is, the translation of one computer language into another language. This could happen multiple times until the instructions are reduced to a language called *assembler* or *assembly*; this code matches closely the architecture of the computer executing the program. For instance, Java programs are compiled into a language called Java bytecode, which the Java Virtual Machine translates into assembler.

3.2.5 Python Installation

Refer now to Chapter 1 for complete installation instructions for Python, Visual Studio Code, and all required tools.

Being an interpreted language, it means that Python requires an *interpreter*. This is what you install when you “*install Python*”. A Python program is simply a text file that contains instructions the interpreter can understand one line (or block of lines) at a time. Since Python is open source, it has many contributors; this results in a robust ecosystem, but the drawback is that there are numerous software libraries that have to be installed while tracking a complex web of co-dependencies. Hence, there is some difficulty in installing a working Python environment to conduct quantitative analysis.

An easy way to use the Python language is through an *Integrated Development Environment*. There are many IDEs that can execute Python code. Being familiar with more than one environment is important as each has strengths and weaknesses. Furthermore, it is a common occurrence that Python programs appear to fail in one environment but work properly in another; this, of course, is a matter of configuration, but it can be a source of frustration in absence of awareness of the potential sources of conflicts in software. Particularly, you want to know how to use tools frequently used in software development, and you need exposure to the multiple options you will find in academia, government and industry.

3.2.6 Executing Python Commands

Commands in Python are case-sensitive, that is, you must respect the capitalization of instructions as they appear in the documentation. The Python interpreter offers multiple mechanisms to interact with the environment. The most commonly used is the Python command line. You can start it from a command shell. Simply type `python`. The command line prompt will change to `>>>`

You can enter Python commands in the command line prompt. Press *Enter* or *return* to execute commands. Like most other programming languages, Python provides mathematical expressions. The building blocks of expressions are variables, numbers, operators, scripts and functions

The basic operations are listed below.

Symbol	Operation
+	Addition
−	Subtraction
★	Multiplication
/	Division
★★	Power

Some commands are just like you would type things in a calculator. If you want to refer the result of the command, you need to give it a name, or in other words, assign the result to a variable. The following command assigns the result to the variable `s`. (Note that Python didn't save the true answer $7/12$ but rather a decimal approximation.)

```
1 >>> s = 1/3 + 1/4
```

If you want to know what the variable `s` contains, simply type its name in the command prompt:

```
1 >>> s
2 0.5833333333333333
```

If a command does not fit on one line, use a backslash (`\`) followed by Return/Enter to indicate that the statement continues on the next line. For example,

```
1 >>> s = 1 - 1/2 + 1/3 - 1/4 + 1/5 - 1/6 + 1/7 \
2 ... :- 1/8 + 1/9 - 1/10 + 1/11 - 1/12
```

Type `s` and Enter to see the new value of `s`.

```
1 >>> s
2 0.6532106782106782
```

Use the double star (`**`) to raise something to an exponent.

```
1 >>> 89**2
2 7921
```

Imaginary numbers can be used with the constant `j`, which stands for $\sqrt{-1}$ by default.

```
1 >>> (1j)**2
2 (-1+0j)
```

Hence, you must be careful in using the variable `j` when dealing with complex numbers. What do you expect from the following operation? Can you explain what you observe? You might need to complete the entire chapter before you can answer this question. **Record your explanation as answer #1.**

```
1 >>> j = 10
2 >>> j = j*(-1)**(0.5)
3 >>> j
```

Does the result stored in the variable `j` match your expectation? Explain.
 Record your explanation as answer #2.

Python uses conventional decimal notation, with an optional decimal point and leading plus or minus sign, for numbers. Scientific notation uses the letter `e` or `E` to specify a power-of-ten scale factor. Imaginary numbers use `i` as a suffix. (This course does not use complex numbers, but you might generate errors with one.)

Some examples of legal numbers are: `1`, `-1`, `0.0001`, `9.87654321`, `1.2345e10`, `1.2345*10**10`, `1j`, `-2j`, `3e5j`.

When assigning names to variables or expressions, it is good practice to make the names useful. For example, in the second command above, we gave the result the name `rhoSq` to imply “rho squared”. Observe that you **CAN NOT** have spaces in variable names (i.e. no spaces in names on the left hand side of the equals sign). Giving useful names to your variables or output will help you keep track of what you are doing.

You can use the up and down arrow keys to easily recall and then edit a command with the left and right arrows. Try it. This is the fastest way to repeat Python commands you might have used in the past.

3.2.7 The VSC Python Editor: Executing A Simple Program

Download the file `lab1SimpleInstruction.py`. Open the file in VSC. The following instructions will appear in the editor:

```
1 k=0;
2 for i in range(1,11):
3     k=k+1/10;
4     print(k)
5 print(k==1)
```

You can click the *Run file* button on the toolbar (as shown in Figure 1), or press `F5`. We call all these actions *to execute a program*. The program `lab1SimpleInstruction.py` adds ten times the number `0.1`. The comparison of the number `1` against the sum returns a value of *false*, whereas elementary arithmetic tells us that adding the number `0.1` ten times should result in `1`, and the returned value should be *true*. Why is this result false? Record your explanation as answer #3. Once the program is executed, the bottom portion of VSC shows a panel labeled **TERMINAL**, as well as other tabs labeled **PROBLEMS**, **OUTPUT**, and **DEBUG CONSOLE**, as shown in Figure 2

Note that you can enter commands in the terminal. For example, type `1+1` and hit enter. Try mathematical operations such as square root, or trigonometric functions. What challenges do you encounter? Record your explanation as answer #4.

In the program `lab1SimpleInstruction.py` all lines of code were executed sequentially, one at a time. This is what we call a **script file**. That is, when

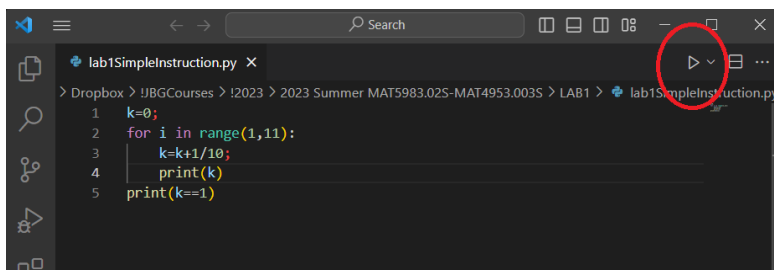


Figure 1: To execute a Python file in VSC, simply click on the Run button, represented by a triangle pointing to the right. It has been highlighted with a circle.

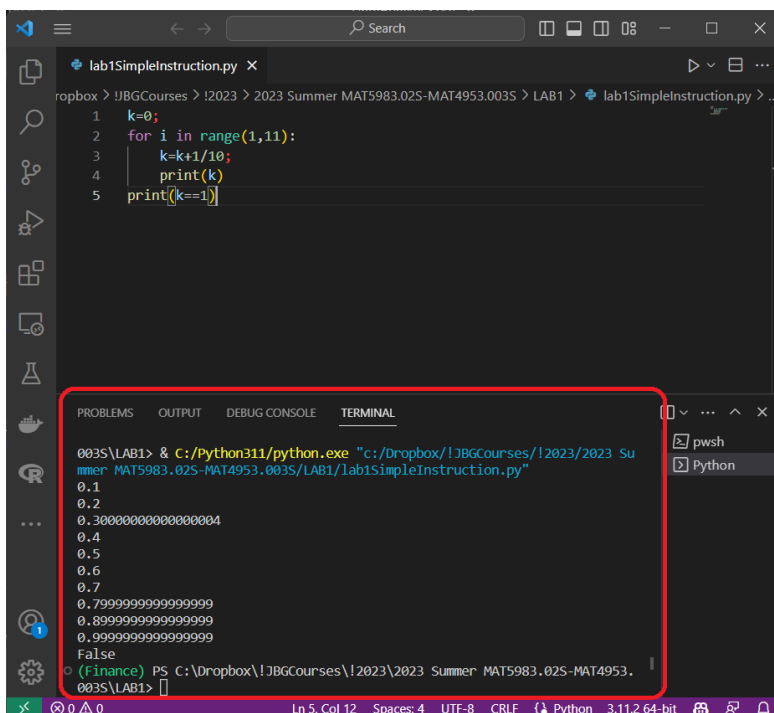


Figure 2: Executing a python program activates the command line. Results are printed in that panel. The command line is highlighted by the round box.

you execute a script file, you expect something tangible to happen, whether it is a message on the screen, a file altered, communication taking place, etc.

There is a way to use source code files to hold code that will be executed later, without necessarily executing any instruction. At this point, open the file `lab1Function.py`. You will see the following instructions:

```
1 def sum1(n):
2     # sum1(n) computes the sum of 1 + 2 + ... + n
3     total = 0;
4     for i in range(1,n):
5         total = total + i
6     return total
```

This file defines a function named `sum1(n)`. The command `sum(3)` returns 6 since $1 + 2 + 3 = 6$. It is instructive to see how this code does its work.

- The instruction `def` indicates the beginning of a *function*. Note that all instructions that are executed inside that function have a vertical alignment

shifted to the right to indicate that all these commands are subordinate to `def`. This shift in vertical alignment is called indentation. It is part of the syntax of the Python programming language, therefore you must always treat indentations strictly.

- The instruction `# sum1(n)...` is a *comment*. Lines of code that start with the symbol `#` are not executed. The semicolon at the end is optional.
- The instruction `total = 0;` is an *assignment*. This line of code creates the variable `total` and assigns to it the value zero.
- The instruction `for i in range(1,n):` starts a *loop*. The subordinate instructions, identified by indentation, will be repeated n times. Particularly, the instruction `range(1,n)` creates a sequence of integers starting in 1 and ending in n . What is the outcome of `range(10,13)` ?
- The instruction `total = total + i` is an assignment. It is very important to emphasize that this is not an equation; if it were, `total` would cancel and `i` would be zero. What takes place is that for every repetition of the loop, the variable `total` is redefined to contain its previous value plus the value of `i`.
- The instruction `return total` is the output of the function. That is, if somehow we could execute this function like `x = sum1(3)`, the variable `x` would contain the value 6, because $\text{sum1}(3) = 1 + 2 + 3 = 6$.

Now, we will modify this file. Append at the end a program called `sum2` in which we sum integers squared. The next question has to be: How do we execute these functions?

The command `python [filename]` can be used to execute a Python program from the terminal, or any command line prompt. But if we run the following in the command prompt, nothing happens:

```
1 python lab1Function.py
```

Why? **Record your explanation as answer #5.**

Now, add the following line of code to the end of the file, **without indentation**:

```
1 print(sum1(2))
```

Execute the program. What was the result of `sum1(2)`? Is this what you expected? Why? **Record your explanation as answer #6.** Compute by hand what the output of `sum1(3)` should be. Now change the last line in the program to read `print(sum1(2))`

What we did was simply to define a function and then use it. Note that if you place the previous line of code at the beginning of the file, the Python interpreter would raise an exception because the function `sum1` is not known to the interpreter yet... keep in mind that the Python interpreter executes one line of code at a time

in sequential order (perhaps with branching and repetition, as we will explore later). Therefore, functions must be defined before they can be invoked within a file.

Did the program produce the result you were expecting? In case it did not, enter the following instruction in the terminal: `man range`. This command shows the manual (`man`) for the function `range`. The fact that `range` behaves in certain ways does not force you to abide by that particular design. In fact, you can modify `sum1` so that the outcome corresponds to what a naive user would expect. To achieve this, you will need to change the limit of range inside the function. Implement it and explain. **Record your explanation as answer #7.**

3.2.8 Executing Functions

In the previous section, we added a single line of code at the end of the program to compute the function we programmed. However, there is a more interesting use case: Create a function that other programs can reuse. In order to use an existing function from a file, we must import that function into the Python environment. We already did something like this when we imported NumPy.

We will now import the function `sum1` from the file `lab1Function.py`. To do this, execute the following command:

```
1 import lab1function as LAB1
```

This instruction loads into memory a directory of all functions present in that file. Now you can invoke the functions you created. For example,

```
1 LAB1.sum2(3)
```

It is also possible to load into memory a single function from a file. We can accomplish this by executing

```
1 from lab1function import sum1
```

In this case, there is no alias associated with the function. To use, you can simply call the function with a parameter, as in `sum2(3)`

Do some research on how to load multiple functions with a single instruction. Explain. **Record your explanation as answer #8.**

3.3 Introduction to NumPy

Numerical operations can be programmed in Python. A library called `NumPy` encapsulates much needed functionality and has become the *de facto* standard. The basic data element in `NumPy` is a multidimensional array. This allows you to solve many technical computing problems, especially those with matrix and vector formulations, in a fraction of the time it would take to write a program in a low-level language such as C.

For example, at the Python prompt type in the following command and press Enter.

```
1 import numpy as np
```

This command loads into memory a reference to the NumPy library. Now you can access all functions in this library by using the prefix `np`; this is an *alias*. You can pick any alias you want, but it is customary to use `np` for NumPy. Now enter the following command:

```
1 np.sqrt(9)
```

The function `sqrt` computes the square root of a real number. Here is an example, and the resulting values.

```
1 >>> rho = (1+np.sqrt(5))/2
2 >>> rhoSq = rho**2
3 >>> a = rhoSq + rho
4 >>> a
5 4.23606797749979
```

However, you cannot use the function `sqrt` with negative numbers. What would you have to do to allow Python to compute the square root of a negative number and return a complex number? **Record your explanation as answer #9.**

The library NumPy offers easy access to commonly used constants, such as:

Constant	Value
<code>np.e</code>	Returns the number e
<code>np.pi</code>	Returns the number π
<code>np.inf</code>	Returns the number ∞
<code>np.nan</code>	NaN, not-a-number

Expressions use the arithmetic operators and precedence rules you already know by now. There are also functions, such as cosine or sine. You surround the input to the function by parenthesis. With NumPy, we can now access commonly used mathematical functions. Among others:

Function	Operation
<code>np.abs(x)</code>	Absolute value $ x $
<code>np.sqrt(x)</code>	Square root $\sqrt{x}$
<code>np.exp(x)</code>	Exponential function e^x
<code>np.log(x)</code>	$\log(x)$ where x is positive real number
<code>np.sin(x)</code>	$\sin(x)$ where x is assumed to be in radians
<code>np.cos(x)</code>	$\cos(x)$ where x is assumed to be in radians

The library NumPy has *many* functions. A comprehensive list can be found at <https://numpy.org/doc/stable/reference/>. Now enter the following command:

```
1 x = np.random.standard_normal(3)
```

There is no output. Why do you think this is the behavior? **Record your explanation as answer #10.** On the right-hand side of the Spyder window there

is a tab called *Variable Explorer*. It will show the value of x . Alternatively, you can enter

```
1 print(x)
```

Alternatively, you can print the result directly without assigning it to a variable. Try

```
1 print(np.random.standard_normal(3))
```

Why are the results of this last operation different to the values stored in variable x ? **Record your explanation as answer #11.**

3.3.1 Matrix Operations

The basic data element in NumPy is a multi-dimensional array. Special meaning is sometimes attached to 1-by-1 matrices, which are called scalars, and to matrices with only one row or column, which are called vectors. Python has other ways of storing both numeric and non-numeric data, but in the beginning, it is usually best to think of everything as a matrix.

Row vectors are delimited in notation by square brackets. A comma separates individual numbers in a row vector. Multiple row vectors of the same size are separated by commas, and are delimited by square brackets to define a matrix. It is common practice to use a capital letter when naming matrices in order to distinguish them from a single vector, scalar or variable value.

Now, we will create a vector and a matrix to demonstrate simple matrix operations

Function	Operation
<code>x = np.array([1,2,3,4])</code>	Creates the row vector $\mathbf{x} = [1, 2, 3, 4]$
<code>b = np.array([1,2,3])</code>	Creates the row vector $\mathbf{b} = [1, 2, 3]$
<code>A = np.array([[1,2,3],[4,5,6],[7,8,9]])</code>	Creates the matrix $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$
<code>B = np.ones([2,3])</code>	Creates the matrix $A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$
<code>C = np.zeros([2,3])</code>	Creates the matrix $A = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

The following commands demonstrate basic vector manipulation cases:

Operation	Outcome
<code>x[-1]</code>	Returns 4, the last element of x
<code>x[1]</code>	Returns 2, the second element of x
<code>x[-2]</code>	Returns 3, the second to last element of x

The colon (`:`) operator is notable. In vector notation, it indicates a range of rows or columns. In Python, the first element of a vector has index 0; the second element has index 1, and so on. The colon operator $a : b$ starts at the row or column of index a and covers up to an index less than b . Thus, `A[0 : 2, 0 : 2]` returns a matrix composed of rows 1 through 2, and columns 1 through 2 with respect to the matrix A .

The following commands demonstrate basic matrix manipulation cases:

Operation	Outcome
<code>A[1,2]</code>	Value of 2 st row and 3 rd column of A
<code>A[0:2,0:2]</code>	Returns $A = \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$
<code>A[1:3,1:3]</code>	Returns $A = \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix}$

The matrix A has three rows and three columns. Then, why does $A[1:100,1:100]$ not produce an error? **Record your explanation as answer #12.**

The size or dimension of a matrix refers to the number of rows and number of columns a matrix has. The `shape` command returns the number of rows and columns of a matrix. Notice that the result is a vector - a one-dimensional matrix with 2 values where the first is the number of rows and the second value is the number of columns. By accessing each element of this vector, we can access each dimension individually.

Using the matrix A of our previous examples,

```
1 >>> y = A.shape()
2 >>> y
3 (3,3)
4 >>> y[0]
5 3
6 >>> y[1]
7 3
```

The following example will illustrate a subtlety in copying matrices:

```
1 >>> D = A
2
3 array([[1, 2, 3],
4        [4, 5, 6],
5        [7, 8, 9]])
6 >>> D[0,0] = 9
7 >>> A
8
9 array([[9, 2, 3],
10        [4, 5, 6],
11        [7, 8, 9]])
```

Note that by updating a value in D , the corresponding value in A was changed. Why? What would you need to do to prevent an update in D to change A ? **Record your explanation as answer #13.**

The basic matrix operations are listed below, using the matrices in the previous examples.

Operation	Outcome
$A.T$	Transpose of A .
$A.conj().T$	Conjugate transpose of A .
$A@B.T$	Matrix multiplication $A \cdot B$.
$A \star A$	Element-wise multiplication
A/A	Element-wise division
$A \star \star 2$	Element-wise exponentiation

NumPy gives an easy way of solving systems of linear equations, Given the matrix A and the vector $\mathbf{b}$ from the previous examples, the linear system $A\mathbf{x} = \mathbf{b}$ can be solved by invoking `np.linalg.solve` as follows

```
1 >>> x = np.linalg.solve(A,b)
2 array([-0.23333333,  0.46666667,  0.1         ])
```

It is easy to use Matrix multiplication $A\mathbf{x}$ to check the answer

```
1 >>> A@x
2 array([1.,  2.,  3.] )
```

Chapter 4

Python Virtual Environments and Workflow Automation

Before we write any scripts, we need to understand *what* those scripts will manage: Python virtual environments.

The Problem: Package Conflicts. Imagine you have two projects on your computer. Project A uses version 1.5 of a library called `numpy`; Project B requires version 2.0. If you install both versions globally, they will conflict. The last one installed wins, and one of your projects could break.

The Solution: Virtual Environments. A **virtual environment** (or *venv*) is an isolated copy of Python with its own set of installed packages, completely separate from every other environment on your machine. Think of it as a self-contained bubble around your project.

- Each project gets its own environment
- Packages installed in one environment cannot interfere with another
- You can recreate an environment exactly from a list of packages
- Deleting an environment is safe and clean

We will cover two strategies for managing environments: The traditional approach via `pip` and an emerging trend called `uv`.

4.1 Traditional Package Management: `pip`

4.1.0.1 Windows (CMD or PowerShell)

```
1 :: Navigate to your project folder
2 cd C:\Projects\DAIR3
3
4 :: Create the environment (Python must be installed
   ↳ globally)
5 python -m venv C:\penv\DAIR3
```

Listing 4.1: Creating a `venv` on Windows

4.1.0.2 Linux / WSL

```
1 # Navigate to your project folder
2 cd ~/DAIR3
3
4 # Create the environment
5 python3 -m venv ~/penv/DAIR3
```

Listing 4.2: Creating a venv on Linux/WSL

In both cases, `C:\penv\DAIR3` or `~/penv/DAIR3` is where the environment will be stored. Keeping all environments in a single folder (`penv` stands for “Python environments”) makes them easy to find and manage.

4.1.1 Activating a Virtual Environment

Activating an environment modifies your current shell session so that `python` and `pip` refer to the isolated copies inside the venv, not the global ones.

4.1.1.1 Windows

```
1 C:\penv\DAIR3\Scripts\activate.bat
```

Listing 4.3: Activating a venv on Windows

After activation, your prompt will change to show the environment name:

```
1 (DAIR3) C:\Projects\DAIR3>
```

4.1.1.2 Linux / WSL

```
1 source ~/penv/DAIR3/bin/activate
```

Listing 4.4: Activating a venv on Linux/WSL

After activation:

```
1 (DAIR3) drg@machine:~/DAIR3$
```

Warning

Note the `source` keyword in the Linux command. This is critical and is explained in detail in Section 4.4.

4.1.2 Installing Packages

Once the environment is active, use `pip` to install packages. The syntax is identical on both platforms:

```
1 pip install numpy pandas matplotlib openai langchain
```

Listing 4.5: Installing packages with pip

To save a list of all installed packages (so you or a colleague can recreate the environment later):

```
1 pip freeze > requirements.txt
```

To recreate an environment from such a list:

```
1 pip install -r requirements.txt
```

4.1.3 Deactivating

To leave the environment and return to the global shell (identical on both platforms):

```
1 deactivate
```

4.1.4 Installing the DAIR3 Required Libraries

With the DAIR3 environment active, install the required libraries for the data challenge:

```
pip install numpy pandas matplotlib scipy jupyter
```

These five libraries are sufficient for all analysis in the data challenge. Additional libraries required for the LLM chapter (including `openai`, `anthropic`, and `langchain`) are installed separately in Chapter 8.

To verify the installation:

```
python -c "import numpy, pandas, matplotlib, scipy, jupyter; print('OK')"
```

You know your core environment is fully functional when:

- `python --version` returns a version number.
- `code .` opens Visual Studio Code from a terminal.
- `git --version` returns a version number.
- The five required libraries import without errors (see above).

The LLM agent programs (`agentGroq.py`, `agentGPT.py`, `agentClaude.py`) are located at `resources/unit_7/F00/` in the DAIR3 repository. Running them requires API keys configured in Chapter 8.

4.2 Modern Package Management: `uv`

4.2.1 Why `uv` Matters Today

The tools introduced in the previous section (`python -m venv` for environment creation and `pip` for package installation) are the established, stable foundation of Python package management. They are maintained by the Python Packaging Authority (PyPA) under the umbrella of the Python Software Foundation (PSF), a non-profit organisation with a long-term governance structure and no commercial dependency. Every Python installation includes `pip` by default, and knowing how to use it remains an essential skill regardless of what other tools you adopt.

A newer tool called `uv` has emerged and is rapidly gaining adoption in professional environments. Understanding both tools is important precisely because the landscape is in transition.

Information

A note on governance. `pip` is maintained by the Python Packaging Authority under the Python Software Foundation, a non-profit with a community governance model and no commercial dependency. `uv` is an open-source tool developed and maintained by Astral, a venture-capital funded commercial company. Astral has been an excellent steward of the tool and has contributed meaningfully to the broader Python packaging ecosystem. Nevertheless, its long-term trajectory depends on business outcomes in a way that foundation-backed projects do not. Until the ecosystem stabilises around a clear community standard, knowing both `pip` and `uv` is the prudent professional position.

4.2.2 What `uv` Offers

`uv` is a drop-in replacement for `pip` and `python -m venv`, rewritten in Rust for performance. Its principal advantages over the traditional toolchain are:

- **Speed.** Package resolution and installation are 10–100 times faster than `pip` due to a Rust implementation and aggressive parallel downloading.
- **A central cache.** Downloaded packages are stored once in a global cache and reused across environments. On the same filesystem, `uv` uses hardlinks so that multiple environments sharing a package consume no additional disk space beyond the first copy.
- **Lockfiles.** `uv` produces a `uv.lock` file that records the complete, resolved dependency graph with cryptographic hashes. This is a significant improvement over `pip freeze > requirements.txt`, which records only installed package names and versions without distinguishing direct dependencies from transitive ones, and without hash verification.

- **pyproject.toml integration.** `uv` works natively with `pyproject.toml`, the modern standard for declaring project dependencies, replacing the older `requirements.txt` workflow.
- **Python version management.** `uv` can download and manage multiple Python versions independently, removing the need for separate tools such as `pyenv`.

4.2.3 Installing uv

4.2.3.1 Windows (PowerShell)

```
1 powershell -ExecutionPolicy Bypass -c "irm
  ↪ https://astral.sh/uv/install.ps1 | iex"
```

Listing 4.6: Installing uv on Windows

`uv` installs itself to `%USERPROFILE%\.local\bin` and adds itself to your user `PATH` automatically. Close and reopen your terminal, then verify:

```
1 uv --version
```

4.2.3.2 Linux / WSL

```
1 curl -LsSf https://astral.sh/uv/install.sh | sh
```

Listing 4.7: Installing uv on Linux/WSL

`uv` installs to `~/.local/bin`. Reload your shell configuration and verify:

```
1 source ~/.bashrc
2 uv --version
```

4.2.4 Creating and Managing Environments with uv

The environment structure produced by `uv venv` is identical to a standard `python -m venv` environment. The same `Scripts\activate.bat` (Windows) and `bin/activate` (Linux) scripts are present and activation is unchanged.

4.2.4.1 Creating an Environment

```
1 uv venv C:\uvenv\DAIR3 --python 3.12
```

Listing 4.8: Creating a uv-managed environment (Windows)

```
1 uv venv ~/uvenv/DAIR3 --python 3.12
```

Listing 4.9: Creating a uv-managed environment (Linux/WSL)

The `--python` flag specifies the Python version. If that version is not already installed, `uv` will download and install it automatically.

4.2.4.2 Activating an Environment

Activation is identical to a standard `venv`:

```
1 C:\uvenv\DAIR3\Scripts\activate.bat
```

```
1 source ~/uvenv/DAIR3/bin/activate
```

4.2.4.3 Installing Packages

Within an activated environment, `uv pip install` replaces `pip install`:

```
1 uv pip install numpy pandas matplotlib
```

Listing 4.10: Installing packages with `uv`

4.2.4.4 Declaring Dependencies with `pyproject.toml`

For projects that will be shared or reproduced, `uv` works best with a `pyproject.toml` file in the project root that declares dependencies explicitly:

```
1 [project]
2 name = "my-project"
3 version = "0.1.0"
4 requires-python = ">=3.12"
5 dependencies = [
6     "numpy>=1.26",
7     "pandas>=2.2",
8     "matplotlib>=3.9",
9 ]
```

Listing 4.11: Minimal `pyproject.toml`

With this file present, a single command installs all dependencies and generates the lockfile:

```
1 uv sync
```

`uv sync` reads `pyproject.toml`, resolves the full dependency graph, installs all packages, and writes `uv.lock`. Running `uv sync` again on another machine, or after a `git pull` that added new dependencies, reproduces the environment exactly.

Information

Commit `pyproject.toml` and `uv.lock` to your repository. Do not commit the environment folder itself. Add it to `.gitignore`.

4.2.5 requirements.txt vs. uv.lock

Table 4.1: Comparison of requirements.txt and uv.lock

Property	requirements.txt	uv.lock
Generated by	pip freeze	uv sync / uv lock
Records	Installed packages only	Full dependency graph
Distinguishes direct vs. transitive deps	No	Yes
Hash verification	No	Yes
Cross-platform reproducibility	Partial	Yes
Should be committed to git	Yes	Yes

4.3 Level 1: Script Files

Now that we understand virtual environments, we can write scripts to activate them automatically. The goal is to create two scripts:

1. **ActivateEnv**: A reusable script that accepts two parameters (environment name and project directory), activates the environment, and opens VS Code.
2. **DAIR3**: A project-specific one-liner that calls **ActivateEnv** with the correct parameters for the DAIR3 project.

4.3.1 The Windows Solution: Batch Files

4.3.1.1 ActivateEnv.bat

The Windows version, **ActivateEnv.bat**, is stored in `C:\penv` and placed on the system `PATH`. It accepts two parameters: `%1` (environment name) and `%2` (project directory).

```

1 @echo off
2 setlocal
3
4 :: Read parameters
5 set "ENV_NAME=%~1"
6 set "BASE_DIR=%~2"
7 echo Starting activation of environment '%ENV_NAME%'...
8
9 :: Validate input
10 if "%ENV_NAME%"==" " (
11     echo [ERROR] Environment name not specified.
12     goto :eof
13 )

```

```

14 if "%BASE_DIR%"==" " (
15     echo [ERROR] Base directory not specified.
16     goto :eof
17 )
18
19 :: Check if base directory exists
20 if not exist "%BASE_DIR%" (
21     echo [ERROR] Directory not found: %BASE_DIR%
22     goto :eof
23 )
24
25 :: Change to working directory
26 cd /d "%BASE_DIR%"
27
28 :: Define activation script path
29 set "VENV_ROOT=C:\penv"
30 set "VENV_ACTIVATE=%VENV_ROOT%\%ENV_NAME%\Scripts\activate.bat"
31
32 :: Activate the environment
33 if exist "%VENV_ACTIVATE%" (
34     echo Activating...
35     call "%VENV_ACTIVATE%"
36     echo Activation complete.
37 ) else (
38     echo [ERROR] Not found: %VENV_ACTIVATE%
39     goto :eof
40 )
41
42 :: Open VS Code in the project directory
43 echo Launching VS Code...
44 code .
45
46 endlocal

```

Listing 4.12: ActivateEnv.bat: Reusable environment launcher (Windows)

Key Windows-specific elements:

- `@echo off` suppresses the echoing of each command to the screen.
- `setlocal` ensures that variable changes are local to this script.
- `%~1` strips surrounding quotes from the first argument.
- `call` is used to invoke another batch file and return control afterward.
- `goto :eof` jumps to the end of the file (a clean exit).

4.3.1.2 DAIR3.bat

This file lives in the DAIR3 project directory. It simply calls `ActivateEnv.bat` with the correct arguments:

```

1 ActivateEnv DAIR3 "C:\Projects\DAIR3"

```

Listing 4.13: DAIR3.bat: Project launcher (Windows)

Because `C:\penv` is on the system PATH, Windows finds `ActivateEnv.bat` automatically. You can run `DAIR3.bat` by double-clicking it or typing `DAIR3` in any CMD window.

4.3.2 The Linux Solution: Shell Scripts

4.3.2.1 Understanding Shell Script Syntax

A Linux shell script is a plain text file containing Bash commands. Every script should begin with a **shebang line**, a special first line that tells the system which interpreter to use:

```
1 #!/bin/bash
```

Parameters passed to a script are accessed as `$1`, `$2`, `$3`, etc. The script's own name is `$0`.

4.3.2.2 ActivateEnv.sh

```
1 #!/bin/bash
2 # ActivateEnv.sh
3 # Usage: source ~/penv/ActivateEnv.sh <ENV_NAME> <PROJECT_DIR>
4 # Note: Must be *sourced*, not executed. See explanation below.
5
6 ENV_NAME="$1"
7 BASE_DIR="$2"
8
9 echo "Starting activation of environment '${ENV_NAME}'..."
10
11 # Validate input
12 if [ -z "$ENV_NAME" ]; then
13     echo "[ERROR] Environment name not specified."
14     return 1
15 fi
16
17 if [ -z "$BASE_DIR" ]; then
18     echo "[ERROR] Base directory not specified."
19     return 1
20 fi
21
22 # Check if the project directory exists
23 if [ ! -d "$BASE_DIR" ]; then
24     echo "[ERROR] Directory not found: ${BASE_DIR}"
25     return 1
26 fi
27
28 # Change to the project directory
29 cd "$BASE_DIR" || return 1
30
31 # Build the path to the activation script
32 VENV_ROOT="$HOME/penv"
33 VENV_ACTIVATE="${VENV_ROOT}/${ENV_NAME}/bin/activate"
34
```

```
35 # Activate the environment
36 if [ -f "$VENV_ACTIVATE" ]; then
37     echo "Activating..."
38     # shellcheck source=/dev/null
39     source "$VENV_ACTIVATE"
40     echo "Activation complete."
41 else
42     echo "[ERROR] Not found: ${VENV_ACTIVATE}"
43     return 1
44 fi
45
46 # Open VS Code in the project directory (WSL-aware)
47 echo "Launching VS Code..."
48 code .
```

Listing 4.14: ActivateEnv.sh: Reusable environment launcher (Linux/WSL)

Notice several differences from the Windows version:

- `-z` tests whether a string is empty (zero length).
- `-d` tests whether a path is a directory.
- `-f` tests whether a path is a regular file.
- `$HOME` is the Linux equivalent of `%USERPROFILE%`: it expands to your home directory.
- `return 1` exits the script with an error code (we use `return` rather than `exit` for a reason explained in the next section).

4.3.2.3 DAIR3.sh

```
1 #!/bin/bash
2 # DAIR3.sh
3 # Usage: source ~/DAIR3/DAIR3.sh
4 # Launches the DAIR3 project environment.
5
6 source ~/penv/ActivateEnv.sh DAIR3 "$HOME/DAIR3"
```

Listing 4.15: DAIR3.sh: Project launcher (Linux/WSL)

4.3.3 Making a Script Executable

On Linux, creating a file does not automatically make it executable. You must grant execute permission using the `chmod` command:

```
1 chmod +x ~/penv/ActivateEnv.sh
2 chmod +x ~/DAIR3/DAIR3.sh
```

`chmod` stands for *change mode*. The `+x` flag adds execute (`x`) permission. Linux permissions are organized around three roles (owner, group, and others), but for personal scripts, `chmod +x` is almost always all you need.

4.3.4 Adding Your Scripts to PATH

On Windows, you added `C:\penv` to the system PATH so that `ActivateEnv.bat` could be found from anywhere. On Linux, you do the same by editing your shell configuration file.

```

1 # Open your bashrc file in a text editor
2 nano ~/.bashrc
3
4 # Add this line at the bottom:
5 export PATH="$HOME/penv:$PATH"
6
7 # Save and reload the configuration
8 source ~/.bashrc

```

Listing 4.16: Adding `~/penv` to PATH in `~/.bashrc`

After this, the shell can find `ActivateEnv.sh` from any directory.

4.3.5 The uv Approach: ActivateUEnv

The following scripts mirror `ActivateEnv` but target `uv`-managed environments stored in `C:\uenv` (Windows) or `~/uenv` (Linux). Store all `uv` environments in `uenv` rather than `penv` to keep the two toolchains visually distinct on disk.

Two behaviours distinguish `ActivateUEnv` from its `pip`-based counterpart:

1. **Automatic creation.** If the named environment does not yet exist, `uv venv` creates it before activation. There is no separate setup step for new projects.
2. **Automatic synchronisation.** If a `pyproject.toml` is present in the project directory, `uv sync` runs after activation. Dependencies install themselves after every `git pull` that adds new requirements, so no manual `pip install` is needed.

4.3.5.1 ActivateUEnv.bat (Windows)

Store this file in `C:\uenv` and add that folder to the system PATH.

```

1 @echo off
2 setlocal
3
4 set "ENV_NAME=%~1"
5 set "BASE_DIR=%~2"
6 set "VENV_ROOT=C:\uenv"
7 set "VENV_PATH=%VENV_ROOT%\%ENV_NAME%"
8 set "VENV_ACTIVATE=%VENV_PATH%\Scripts\activate.bat"
9
10 :: Validate inputs
11 if "%ENV_NAME%"==" " (
12     echo [ERROR] Usage: ActivateUEnv ^<env-name^> ^<project-dir^>
13     goto :eof

```

```

14 )
15 if "%BASE_DIR%"==" " (
16     echo [ERROR] Usage: ActivateUVEnv ^<env-name^> ^<project-dir^>
17     goto :eof
18 )
19 if not exist "%BASE_DIR%" (
20     echo [ERROR] Project directory not found: %BASE_DIR%
21     goto :eof
22 )
23
24 :: Create environment if it does not exist yet
25 if not exist "%VENV_ACTIVATE%" (
26     echo [INFO] Environment '%ENV_NAME%' not found. Creating with
27     ↪ uv...
28     uv venv "%VENV_PATH%" --python 3.12
29     if errorlevel 1 (
30         echo [ERROR] Failed to create environment.
31         goto :eof
32     )
33 )
34
35 :: Activate
36 cd /d "%BASE_DIR%"
37 call "%VENV_ACTIVATE%"
38
39 :: Sync dependencies if pyproject.toml is present
40 if exist "pyproject.toml" (
41     echo [INFO] pyproject.toml found. Syncing dependencies...
42     uv sync
43 )
44
45 echo.
46 echo [OK] Environment '%ENV_NAME%' active.
47 echo Launching VS Code...
48 code .
49
endlocal

```

Listing 4.17: ActivateUVEnv.bat: uv environment launcher (Windows)

Usage is identical in form to ActivateEnv.bat:

```

1 ActivateUVEnv lunar "C:\Dropbox\Research\Lunar\python"
2 ActivateUVEnv forecasting "C:\Projects\forecasting"

```

4.3.5.2 ActivateUVEnv: Linux / WSL Shell Function

Add the following function to `~/.bashrc`. Like its `pip`-based counterpart, it runs in the current shell so that activation and directory changes persist in your session.

```

1 # -----
2 # ActivateUVEnv: Activate a uv-managed venv and open VS Code
3 # Usage: ActivateUVEnv <ENV_NAME> <PROJECT_DIR>
4 # -----
5 ActivateUVEnv() {

```

```

6   local ENV_NAME="$1"
7   local BASE_DIR="$2"
8   local VENV_ROOT="$HOME/uvENV"
9   local VENV_PATH="${VENV_ROOT}/${ENV_NAME}"
10  local VENV_ACTIVATE="${VENV_PATH}/bin/activate"
11
12  if [ -z "$ENV_NAME" ]; then
13      echo "[ERROR] Usage: ActivateUVEEnv <env-name>
14          ↪ <project-dir>"
15      return 1
16  fi
17
18  if [ -z "$BASE_DIR" ]; then
19      echo "[ERROR] Usage: ActivateUVEEnv <env-name>
20          ↪ <project-dir>"
21      return 1
22  fi
23
24  if [ ! -d "$BASE_DIR" ]; then
25      echo "[ERROR] Project directory not found: ${BASE_DIR}"
26      return 1
27  fi
28
29  # Create environment if it does not exist yet
30  if [ ! -f "$VENV_ACTIVATE" ]; then
31      echo "[INFO] Environment '${ENV_NAME}' not found.
32          ↪ Creating with uv..."
33      uv venv "$VENV_PATH" --python 3.12 || return 1
34  fi
35
36  # Activate
37  cd "$BASE_DIR" || return 1
38  # shellcheck source=/dev/null
39  source "$VENV_ACTIVATE"
40
41  # Sync dependencies if pyproject.toml is present
42  if [ -f "pyproject.toml" ]; then
43      echo "[INFO] pyproject.toml found. Syncing
44          ↪ dependencies..."
45      uv sync
46  fi
47
48  echo "[OK] Environment '${ENV_NAME}' active."
49  echo "Launching VS Code..."
50  code .
51 }

```

Listing 4.18: ActivateUVEEnv() shell function for ~/.bashrc

Reload .bashrc after saving:

```
1 source ~/.bashrc
```

Then call it from any directory:

```
1 ActivateUVEEnv lunar "$HOME/Research/Lunar/python"
```

4.4 The Critical Difference: Executing vs. Sourcing

This section explains one of the most important, and most misunderstood, concepts in Linux shell scripting, especially for developers coming from Windows.

4.4.1 Child Shells

Every time you run a script with `./script.sh` or just `script.sh`, Linux launches a **child shell**: a new, separate shell process that inherits a copy of your environment, runs the script, and then *terminates*. Any changes the script makes to the environment (activating a venv, changing directory, setting a variable) exist only in that child shell. When the child terminates, those changes disappear. Your original terminal is completely unaffected.

This is why running `./ActivateEnv.sh` would seem to do nothing useful: the venv gets activated inside the child, then the child exits, and your terminal is still pointing at the global Python.

4.4.2 Sourcing: Running in the Current Shell

The `source` command (or its shorthand, a single dot `.`) runs a script *in the current shell*: no child is created. Every command executes as if you had typed it directly at the prompt. This means:

- `source /penv/DAIR3/bin/activate` activates the venv in *your* terminal.
- `cd` changes *your* working directory.
- Variables set by the script persist in *your* session.

```
1 # These two are equivalent:
2 source ActivateEnv.sh DAIR3 ~/DAIR3
3 . ActivateEnv.sh DAIR3 ~/DAIR3
4
5 # This does NOT work for venv activation:
6 ./ActivateEnv.sh DAIR3 ~/DAIR3    # child shell; changes are
    ↪ lost
```


4.4.3 The Windows Parallel

Windows does not have this distinction in quite the same way. The `call` command in a batch file runs another `.bat` in the same CMD session and returns, which is why the Windows version works as expected. Linux's default behavior of spawning a child process is actually the *safer* design, as it prevents scripts from accidentally modifying your environment, but it requires you to opt in to the shared-session behavior with `source`.

Note

This is also why `ActivateEnv.sh` uses `return` instead of `exit` for error handling. When a script is sourced, `exit` would close your entire terminal. `return` exits only the script, leaving your session intact.

4.5 Level 2: Shell Functions, The Elegant Linux Approach

Sourcing scripts is powerful, but it requires typing `source` or `.` every time. It also means your launcher script must live somewhere on the `PATH` and the user must remember the exact invocation. Linux offers a more elegant solution: **shell functions**.

4.5.1 What Is a Shell Function?

A shell function is like a script, but it is defined directly inside your shell configuration file (`~/.bashrc`). Because it is part of the shell itself (not a separate file), it automatically runs in the current session without requiring `source`. You call it just by typing its name.

4.5.2 Defining the ActivateEnv Function

Add the following to the bottom of `~/.bashrc`:

```
1 # -----
2 # ActivateEnv: Activate a Python venv and open VS Code
3 # Usage: ActivateEnv <ENV_NAME> <PROJECT_DIR>
4 # -----
5 ActivateEnv() {
6     local ENV_NAME="$1"
7     local BASE_DIR="$2"
8
9     echo "Starting activation of environment '${ENV_NAME}'..."
10
11     # Validate input
12     if [ -z "$ENV_NAME" ]; then
13         echo "[ERROR] Environment name not specified."
14         return 1
15     fi
```

```

16
17     if [ -z "$BASE_DIR" ]; then
18         echo "[ERROR] Base directory not specified."
19         return 1
20     fi
21
22     if [ ! -d "$BASE_DIR" ]; then
23         echo "[ERROR] Directory not found: ${BASE_DIR}"
24         return 1
25     fi
26
27     local VENV_ACTIVATE="$HOME/penv/${ENV_NAME}/bin/activate"
28
29     if [ ! -f "$VENV_ACTIVATE" ]; then
30         echo "[ERROR] Virtual environment not found:
31             ↪ ${VENV_ACTIVATE}"
32         return 1
33     fi
34
35     # Change directory, activate, and open VS Code
36     cd "$BASE_DIR" || return 1
37     source "$VENV_ACTIVATE"
38     echo "Environment '${ENV_NAME}' activated."
39 }

```

Listing 4.19: Shell function version of ActivateEnv in ~/.bashrc

The keyword `local` restricts variables to the function's scope, so they do not pollute your shell session.

4.5.3 Defining the DAIR3 Function

Now add a one-line function that calls `ActivateEnv` for the DAIR3 project:

```

1 # -----
2 # DAIR3: Launch the DAIR3 project environment
3 # Usage: DAIR3
4 # -----
5 DAIR3() {
6     ActivateEnv "DAIR3" "$HOME/DAIR3"
7 }

```

Listing 4.20: Project-specific function for DAIR3 in ~/.bashrc

4.5.4 Activating the New Functions

After editing ~/.bashrc, reload it:

```

1 source ~/.bashrc

```

Now, from *any directory*, simply type:

1 DAIR3

The shell will change to your DAIR3 project directory, activate the virtual environment, and open VS Code, all in one word, with no `source` prefix required.

4.5.5 Why Windows Has No Clean Equivalent

Windows batch files always run as child processes; there is no `source` command in CMD. PowerShell comes closest: you can define functions in your PowerShell profile (\$PROFILE), and they behave similarly to Bash functions. However, PowerShell's profile is disabled by default for security reasons and requires changing the execution policy. For Windows users who want this one-command experience natively, configuring PowerShell profiles is the path forward, but it involves more administrative overhead than editing `~/.bashrc`. This is one of the practical reasons many developers on Windows adopt WSL.

4.6 Putting It All Together

Figure 4.2 summarizes the complete workflow on each platform.

Table 4.2: Complete workflow comparison: Windows vs. Linux/WSL

Step	Windows	Linux / WSL
Create venv	<code>python -m venv C:\penv\DAIR3</code>	<code>python3 -m venv ~/penv/DAIR3</code>
Reusable launcher	<code>ActivateEnv.bat</code> in <code>C:\penv</code>	Function <code>ActivateEnv()</code> in <code>~/.bashrc</code>
Project launcher	<code>DAIR3.bat</code> in project folder	Function <code>DAIR3()</code> in <code>~/.bashrc</code>
Make available globally	Add <code>C:\penv</code> to system PATH	Functions in <code>.bashrc</code> are always available
Launch project	Type <code>DAIR3</code> in any CMD window	Type <code>DAIR3</code> in any terminal
Result	venv active, VS Code opens in project folder	venv active, VS Code opens in project folder (WSL mode)

4.6.1 The Full WSL Experience

When you type `DAIR3` in your WSL terminal:

1. Bash looks up `DAIR3` in its list of known functions.
2. `DAIR3()` calls `ActivateEnv "DAIR3" "$HOME/DAIR3"`.

3. `ActivateEnv()` changes to the project directory.
4. The virtual environment is activated in your current shell.
5. `code .` launches VS Code with the WSL extension, connecting your Windows VS Code interface to your Linux backend.
6. You are now working in a fully configured development environment with one word.

4.7 Quick Reference Cheat Sheet

Table 4.3: Shell scripting and virtual environment cheat sheet

Task	Windows (CMD)	Linux / WSL (Bash)
Create virtual environment	<code>python -m venv C:\penv\NAME</code>	<code>python3 -m venv ~/penv/NAME</code>
Activate environment	<code>C:\penv\NAME\Scripts\activate</code>	<code>~/penv/NAME/bin/activate</code>
Deactivate environment	<code>deactivate</code>	<code>deactivate</code>
Install a package	<code>pip install pkg</code>	<code>pip install pkg</code>
Save package list	<code>pip freeze > req.txt</code>	<code>pip freeze > req.txt</code>
Restore from list	<code>pip install -r req.txt</code>	<code>pip install -r req.txt</code>
Make script executable	(not required; extension handles it)	<code>chmod +x script.sh</code>
Run a script	<code>script.bat</code>	<code>./script.sh</code>
Source a script	<code>call script.bat</code>	<code>source script.sh</code>
Edit shell config	(no equivalent in CMD)	<code>nano ~/.bashrc</code>
Reload shell config	(restart CMD)	<code>source ~/.bashrc</code>
Open VS Code (WSL)	<code>code .</code>	<code>code .</code>
Script parameter 1	<code>%~1</code>	<code>\$1</code>
Check if variable empty	<code>if "%VAR%"==" "</code>	<code>if [-z "\$VAR"]</code>
Check if directory exists	<code>if exist "DIR\"</code>	<code>if [-d "DIR"]</code>
Exit a script/function	<code>goto :eof</code>	<code>return (in function/-sourced script)</code>

Table 4.4: uv quick reference: equivalences with pip and venv

Task	pip / venv	uv
Install uv	(not applicable)	See Section 4.2
Create environment	python -m venv PATH	uv venv PATH --python 3.12
Activate environment	source PATH/bin/activate	source PATH/bin/activate
Install a package	pip install pkg	uv pip install pkg
Install from pyproject.toml	pip install -e .	uv sync
Add a dependency	Edit requirements.txt manually	uv add pkg
Remove a dependency	Edit requirements.txt manually	uv remove pkg
Generate lockfile	pip freeze > req.txt	uv lock (or implicit in uv sync)
Install from lockfile	pip install -r req.txt	uv sync
Launcher script (Windows)	ActivateEnv.bat	ActivateUVEnv.bat
Launcher function (Linux)	ActivateEnv() .bashrc	in ActivateUVEnv() in .bashrc
Environment root	C:\penv / ~/penv	C:\uvenv / ~/uvenv

4.8 Summary

This chapter covered the tools and techniques for managing Python environments and automating project setup across Windows and Linux:

- **Python virtual environments:** isolated, per-project copies of Python that prevent dependency conflicts between projects. Each environment has its own set of installed packages, and deleting one has no effect on any other.
- **The standard pip and venv toolchain:** the foundation-backed tools included with every Python installation for creating environments, installing packages, and recording dependencies in `requirements.txt`. Knowing this toolchain remains an essential skill regardless of what other tools you adopt.
- **uv:** a modern, high-performance alternative developed by Astral that offers dramatically faster installs, a global package cache, reproducible lockfiles with cryptographic hashes, and native `pyproject.toml` support. Because `uv` automatically synchronises dependencies from a lockfile via `uv sync`, environment reproduction becomes a single command rather than a manual

process.

- **Level 1 scripting:** writing reusable launcher scripts (`ActivateEnv.bat` and `ActivateEnv.sh` for `pip`-managed environments; `ActivateUVEEnv.bat` and the `ActivateUVEEnv()` shell function for `uv`-managed environments) and project-specific one-liners (`DAIR3.bat`, `DAIR3.sh`) that activate the correct environment, change to the project directory, and launch VS Code in a single invocation.
- **Level 2 scripting:** shell functions defined in `~/.bashrc` that provide a one-word launch command from any directory, with no `source` prefix required.
- **The source vs. execute distinction:** on Linux, running a script normally spawns a child shell whose environment changes vanish when it exits. The `source` command (or its shorthand `.`) runs the script in the current shell, which is why virtual environment activation requires sourcing. This same distinction explains why `ActivateEnv.sh` uses `return` instead of `exit` for error handling.

The `DAIR3` command you built in this chapter is a small but meaningful example of a broader principle: the CLI lets you encode your workflow into repeatable, shareable, and improvable scripts. Every tedious setup step you eliminate today is cognitive load freed up for the work that actually matters.

Chapter 5

Numerical Operations

5.1 Bits, Bytes, and Floating Point

Computers have a finite number of ways to represent numbers. That is, they can only approximate a small subset of real numbers. Understanding this is an essential component of conducting numerical operations: You need to understand how and why numerical errors accumulate, and when they interfere with a calculation.

The unit of computing representation is a **bit** (unit represented by *b*), i.e. a circuit that is either open or closed. Eight bits are grouped into one **byte** (unit represented by *B*), as shown in Figure 1. This number evolved from computer architectures in the early days of computing, and was first standardized by ISO/IEC 2382-1:1993. In one byte, positions go from 0 to 7, usually represented from right to left. A 1 in the first position represents 2^0 , in the second position is 2^1 , and so on.

A naive way to represent numbers with a computer is to allocate N decimal digits, for each number. For example:

$$012.345$$

This example represents a 6-digit fixed-point system. This 6-digit fixed-point system results in errors in some sum cases:

$$999.999 + 999.999 = \text{OVERFLOW}$$

Also errors occur in multiplication and division

$$013.000/003.000 = 004.333$$

In this case, we lose all digits after the third decimal.

We can allow the decimal point to “float” to gain accuracy with the same number of digits. For example:

$$013.000/003.000 = 4.33333$$

We could even represent larger numbers than 6 digits allowed us before:

$$999.999 \times 999.999 = 9.99998 \times 10^5$$

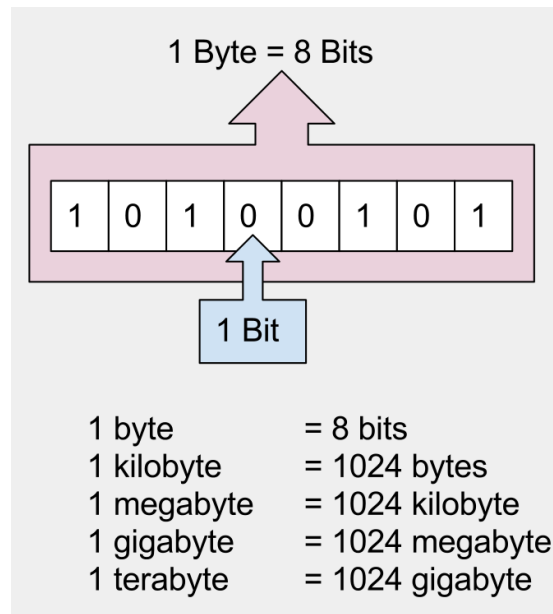


Figure 1: Bits and Bytes [?]. Reading from left to right, the example represents: $2^0 + 2^2 + 2^5 + 2^7 = 1 + 4 + 32 + 128 = 165$. If you read from right to left (this is the standard way to represent bits), you would obtain $2^0 + 2^2 + 2^5 + 2^7 = 165$. In this case, left-to-right and right-to-left produce the same result. But in general they don't have to be the same. What happens if you replace any zero with a one, and compute both directions again? Thus, representation matters, and you need to know the convention.

5.1.1 The IEEE Standard for Floating-Point Arithmetic

The IEEE Standard for Floating-Point Arithmetic (IEEE 754) is a technical standard for floating-point computation established in 1985 by the Institute of Electrical and Electronics Engineers (IEEE).

- Finite numbers may be either base 2 (binary) or base 10 (decimal).
- Each finite number is described by three integers:
 1. s = a sign (zero or one),
 2. c = a significand (or 'coefficient'),
 3. q = an exponent.
- The numerical value of a finite number is $(-1)^s \times c \times b^q$ where b is the base (2 or 10), also called radix.

The three fields in an IEEE-754 float. Binary floating-point numbers are stored in a sign-magnitude form where the most significant bit is the *sign* bit, *exponent* is the biased exponent, and *fraction* is the significand minus the most significant bit. Figure 2 shows a 32-bit number, also known as a single-precision floating number, whereas Figure 3 depicts a 64-bit number, also known as double-precision floating number.

If we want to represent a real number with a finite number N of memory positions, we can reserve one position for the sign, $N - k - 1$ for the integer

Cap. X.

ALGEBRA.

41

variatur. Et quidem Augetur, si majore multiplicatore multiplicetur Numerator quam Denominator: Minuitur, si contra.

Sive (quod eodem recidit)

Si Fractionis duo termini, (Numerator & Denominator,) ita (addendo) augentur ambo, ut augmenta sint ipsis terminis proportionalia; idem qui prius manet fractionis Valor: Augetur vero, si Numeratoris augmentum, ad augmentum Denominatoris, majorem rationem habeat, quam habet ipse Numerator ad Denominatorem: Minuitur, si minorem.

Quodque de Numeratore & Denominatore Fractionis dictum est, similiter intelligendum est de Rationis Antecedente & Consequente, & pariter in sequentibus.

$$\begin{array}{l}
 \text{Putat, } \frac{1}{3} = \frac{1 \times 2}{3 \times 2} = \frac{2}{6} = \frac{3}{9} = \frac{4}{12} \text{ & c. } \quad \frac{n}{d} = \frac{2n}{2d} = \frac{3n}{3d} = \frac{an}{ad} \\
 \text{Sen, } \frac{1}{3} = \frac{1+1}{3+3} = \frac{1+2}{3+6} = \frac{1+3}{3+9} \text{ & c. } \quad \frac{n}{d} = \frac{n+n}{d+d} = \frac{n+2n}{d+2d} = \frac{n+3n}{d+3d} \\
 \text{Sed, } \frac{1}{3} < \frac{1 \times 3}{3 \times 2} = \frac{3}{6} = \frac{1+2}{3+3} \quad \frac{n}{d} < \frac{3n}{2d} = \frac{n+2n}{d+d} \\
 \text{Et, } \frac{1}{3} > \frac{1 \times 2}{3 \times 3} = \frac{2}{9} = \frac{1+1}{3+6} \quad \frac{n}{d} > \frac{2n}{3d} = \frac{n+n}{d+2d}
 \end{array}$$

SOLUTIO.

His premis; si datae fractionis, ad minimos terminos (communi divisione per maximam communem mensuram) reducta, tum Numerator, tum Denominator, dato numero non sit major: fit quod imperatur. Quippe Fractio est in minimis terminis, & qualis requiritur.

Sin Fractionis, sic reducta, terminus vel alter vel uterque sit exposito numero major: querenda est fractio quae sit quam exposita vel proxime major, vel proxime minor, terminos habens exposito numero non majores; ea quae sequitur methode.

Pro Proxime-Majori.

Pro invenienda proxime majori, sic procedo.

Denominatorem fractionis expositae (aut ejus ad quam reducitur, si praecesserit reductio,) per Numeratorem divido; ut habeam, qui Numeratori 1 respondeat, Denominatorem, in integris cum annexis partibus decimalibus, tam accuratum quam videbitur expedire.

Putat, 2684769) 8376571 (3.12003416 +.

Nam, ut 2684769 ad 8376571, sic 1 ad 3.12003416 +.

Adeoquae, pro exposita fractione $\frac{8376571}{2684769}$, hanc habeo ei (quam prope censetur necessarium) aequipollentem $\frac{312003416}{2684769}$ quae numeratorem habet 1. Quam Fractionem primam completam appello; eandemque, neglectis partibus decimalibus, appello fractionem primam truncatam. Ejusque partes decimales abscissas, 0.12003416, Appendicem voco, sive Mantissam fractionis primae, seu Mantissam primam.

Estque haec truncata Fractio $\frac{1}{3}$, tum justo major (propter denominatorem justo minorem, utpote partibus decimalibus mulctatum;) tum proxime-major, omnium habentium numeratorem 1, & denominatorem integrum, (nam $\frac{1}{3}$ est adhuc major, & $\frac{1}{4}$ justo minor, & de reliquis similiter:) Neque, ex justo majoribus, propius ad verum valorem accedere potest ullus, nisi aucto numeratore 1.

F

Loco

Figure 4: Page 41 of "A Treatise of Algebra," where the word mantissa is introduced. J. Wallis. *Opera mathematica: De Algebra Tractatus*. Number v. 5 Anno 1685 Anglice editus; Nunc Auctus Latine. Theatrum Sheldonianum, 1693.

results algebraically in zero. However,

$$1 - 1/3 - 2/3 = 1.1102e - 16,$$

$$1 - 2/3 - 1/3 = 5.5511e - 17.$$

Thus, not only the operation is not zero, but also the order of summands matter, i.e. floating point addition is not commutative. Hence, $1 - 1/3 - 2/3 \neq 1 - 2/3 - 1/3 \neq 0$

Because floating point numbers cannot represent all real numbers, errors of representation occur in many cases. But why is the operation noncommutative?

Record your explanation as answer #14.

The following instruction compares whether $10^{30} + 1 - 10^{30}$ equals the value 1. Using elementary arithmetic, the answer should be a resounding *yes*. In the Python language, the exponent is represented by a double asterisk, like in Fortran, instead of the carat sign (^) used in most other computing platforms. The double equal sign tests whether the two sides are equal, and returns a Boolean value of *true* or *false*. Hence, the question is translated into the following instruction:

```
1 10**30 + 1 - 10**30 == 1
```

This particular instruction returns a Boolean value of *true* in Python and *false* in Matlab (the correct Matlab syntax is $10^30 + 1 - 10^30 == 1$). However, the following statement:

```
1 10**30 + 1. - 10**30 == 1
```

returns *false* in Python. Why? **Record your explanation as answer #15.**

An operation like

```
1 10**20 + 1 - 10**20 == 1
```

returns different results in Matlab, C, Fortran (*false*) vs. Python (*true*). It might be tempting to declare Python the winner in numerical computation, but what truly happens is that Python does not follow the IEEE-754 standard strictly. To learn about more subtleties of floating point in Python, go to <https://docs.python.org/3/tutorial/floatingpoint.html>. In most cases, deviations from the standard in the Python interpreter are harmless and have an ideological origin rather than a technical one. However, it is conceivable that extreme algorithms developed under the IEEE standard would have to be carefully adapted in Python. There are alternatives, but they are beyond the scope of this chapter. For now, it suffices to say that there are landmines in the Python landscape.

We can explore further discrepancies in extreme floating point values by exploring in two numerical environments the results of $10 \cdot 10^b + 1 - 10 \cdot 10^b$ for different values of b . Download and install Python, Matlab, R, or compile a C/C++ program that executes the same operations. Compare what happens in these environments when you choose $b \in \{10, 1e2, 1e10, 1e100, 1e1000\}$. Describe the discrepancies, and analyze. **Record your explanation as answer #16.**

5.2 Introduction to NumPy

To execute mathematical operations beyond basic operations with real numbers, you need to import a numerical library. Matrix operations are executed through functions in NumPy. Hence, the first step is to load NumPy.

```
1 In : import numpy as np
```

The library NumPy offers easy access to commonly used constants, such as:

Constant	Value
np.e	Returns the number e
np.pi	Returns the number π
np.inf	Returns the number ∞
np.nan	NaN, not-a-number

With NumPy, we can now access commonly used mathematical functions. Among others:

Function	Operation
np.abs(x)	Absolute value $ x $
np.sqrt(x)	Square root $\sqrt{x}$
np.exp(x)	Exponential function e^x
np.log(x)	$\log(x)$ where x is positive real number
np.sin(x)	$\sin(x)$ where x is assumed to be in radians
np.cos(x)	$\cos(x)$ where x is assumed to be in radians

Expressions use the arithmetic operators and precedence rules you already know by now. There are also functions, such as cosine or sine. You surround the input to the function by parenthesis. For example, to calculate the square root of two, you type

```
1 In : np.sqrt(2)
2 Out : 1.4142135623730951
```

However, you cannot use the function `sqrt` with negative numbers. What would you have to do to allow Python to compute the square root of a negative number and return a complex number? **Record your explanation as answer #17.**

The library NumPy has *many* functions. A comprehensive list can be found at <https://numpy.org/doc/stable/reference/>. You will observe that immediately below the command you entered, the prompt will show `Out[n]: 3.0`, where `n` stands for the command sequence number (recall, the first executed command has $n = 1$, the second $n = 2$, and so on).

Now enter the following command:

```
1 x = np.random.standard_normal(3)
```

There is no output. Why do you think this is the behavior? **Record your explanation as answer #18.** On the right-hand side of the Spyder window there

is a tab called *Variable Explorer*. It will show the value of x . Alternatively, you can enter

```
1 print(x)
```

Alternatively, you can print the result directly without assigning it to a variable. Try

```
1 print(np.random.standard_normal(3))
```

Why are the results of this last operation different to the values stored in variable x ? **Record your explanation as answer #19.**

5.2.1 Matrix Operations

The basic data element in NumPy is a multi-dimensional array. Special meaning is sometimes attached to 1-by-1 matrices, which are called scalars, and to matrices with only one row or column, which are called vectors. Python has other ways of storing both numeric and non-numeric data, but in the beginning, it is usually best to think of everything as a matrix.

Row vectors are delimited in notation by square brackets. A comma separates individual numbers in a row vector. Multiple row vectors of the same size are separated by commas, and are delimited by square brackets to define a matrix. It is common practice to use a capital letter when naming matrices in order to distinguish them from a single vector, scalar or variable value.

Now, we will create a vector and a matrix to demonstrate simple matrix operations

Function	Operation
$x = np.array([1, 2, 3, 4])$	Creates the row vector $\mathbf{x} = [1, 2, 3, 4]$
$b = np.array([1, 2, 3])$	Creates the row vector $\mathbf{b} = [1, 2, 3]$
$A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])$	Creates the matrix $A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$
$B = np.ones([2, 3])$	Creates the matrix $A = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$
$C = np.zeros([2, 3])$	Creates the matrix $A = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$

The following commands demonstrate basic vector manipulation cases:

Operation	Outcome
$x[-1]$	Returns 4, the last element of x
$x[1]$	Returns 2, the second element of x
$x[-2]$	Returns 3, the second to last element of x

The colon (:) operator is notable. In vector notation, it indicates a range of rows or columns. In Python, the first element of a vector has index 0; the second element has index 1, and so on. The colon operator $a : b$ starts at the row or column of index a and covers up to an index less than b . Thus, $A[0 : 2, 0 : 2]$ returns a matrix

composed of rows 1 through 2, and columns 1 through 2 with respect to the matrix A .

The following commands demonstrate basic matrix manipulation cases:

Operation	Outcome
$A[1, 2]$	Value of 2 st row and 3 rd column of A
$A[0 : 2, 0 : 2]$	Returns $A = \begin{bmatrix} 1 & 2 \\ 4 & 5 \end{bmatrix}$
$A[1 : 3, 1 : 3]$	Returns $A = \begin{bmatrix} 5 & 6 \\ 8 & 9 \end{bmatrix}$

The matrix A has three rows and three columns. Then, why does $A[1 : 100, 1 : 100]$ not produce an error? **Record your explanation as answer #20.**

The size or dimension of a matrix refers to the number of rows and number of columns a matrix has. The **shape** command returns the number of rows and columns of a matrix. Notice that the result is a vector - a one-dimensional matrix with 2 values where the first is the number of rows and the second value is the number of columns. By accessing each element of this vector, we can access each dimension individually.

Using the matrix A of our previous examples,

```
1 In : y = A.shape()
2 In : y
3 Out : (3,3)
4 In : y[0]
5 Out : 3
6 In : y[1]
7 Out : 3
```

The following example will illustrate a subtlety in copying matrices:

```
1 In : D = A
2 Out :
3 array([[1, 2, 3],
4         [4, 5, 6],
5         [7, 8, 9]])
6 In : D[0,0] = 9
7 In : A
8 Out :
9 array([[9, 2, 3],
10        [4, 5, 6],
11        [7, 8, 9]])
```

Note that by updating a value in D , the corresponding value in A was changed. Why? What would you need to do to prevent an update in D to change A ? **Record your explanation as answer #21.**

The basic matrix operations are listed below, using the matrices in the previous examples.

Operation	Outcome
$A.T$	Transpose of A .
$A.conj().T$	Conjugate transpose of A .
$A @ B.T$	Matrix multiplication $A \cdot B$.
$A \star A$	Element-wise multiplication
A / A	Element-wise division
$A \star \star 2$	Element-wise exponentiation

NumPy gives an easy way of solving systems of linear equations, Given the matrix A and the vector $\mathbf{b}$ from the previous examples, the linear system $A\mathbf{x} = \mathbf{b}$ can be solved by invoking `np.linalg.solve` as follows

```
1 In : x = np.linalg.solve(A,b)
2 Out: array([-0.23333333,  0.46666667,  0.1          ])
```

It is easy to use Matrix multiplication $A\mathbf{x}$ to check the answer

```
1 In : A @ x
2 Out: array([1.,  2.,  3.] )
```

5.3 Specific tasks for this lab

The following chapter is a lecture about Principal Component Analysis (PCA), one of the most basic techniques for data analysis. Implement a principal component analysis according to the following procedure (use NumPy to conduct matrix operations):

1. Select weight at birth plus other numerical variables from the NCHS data set. We will call this the *feature matrix*.
2. Ensure that the mean is zero for every variable. How do you achieve this? **Record your explanation as answer #22.** We will call this the *normalized feature matrix*.
3. Compute the inner product of the normalized feature matrix. What is the dimension of the resulting matrix? **Record your explanation as answer #23.**
4. Compute eigenvalues and eigenvectors for the inner product of the normalized feature matrix. You will need to sort eigenvalues by magnitude. How do you do that in Python? **Record your explanation as answer #24.**
5. Project the data in the feature matrix along the eigenvectors corresponding to the two largest eigenvalues. Plot each observation in this projected space according to quartile of weight at birth. You might need to pick different variables until a rough pattern emerges. Interpret the plot. **Record your explanation as answer #25.**

Note that you are not expected to simply call a PCA library.

Chapter 6

Introduction to L^AT_EX

L^AT_EX is a document preparation system in which the author writes plain text interleaved with markup commands, and a compiler transforms that source into a typeset PDF. This contrasts with word processors such as Microsoft Word or Google Docs, where formatting is applied interactively in a what-you-see-is-what-you-get (WYSIWYG) editor. The key property that makes L^AT_EX the standard in science and mathematics is its ability to render arbitrarily complex mathematical notation with precision and consistency. Every document produced in this program is a L^AT_EX document.

6.1 Your First L^AT_EX Document

A L^AT_EX source file is a plain text file with the extension `.tex`. Just as `hello.py` was a plain text file that Python interpreted, `hello.tex` is a plain text file that the L^AT_EX compiler processes into a PDF. The same text editor rules apply (see Section 3.1): use Notepad, TextEdit in plain-text mode, or `gedit`; never a word processor.

Information

Exercise L-1: Your First L^AT_EX Document Open a plain text editor (see Table 3.1), type the following exactly, and save the file as `hello.tex`:

```
1 \documentclass{article}
2 \begin{document}
3
4 Hello, world. This is my first \LaTeX{} document.
5
6 Here is some mathematics:  $e^{i\pi} + 1 = 0$ .
7
8 \end{document}
```

Each line of the listing serves a specific purpose:

- `\documentclass{article}` declares the document class. For most reports and papers, `article` is the correct choice.
- `\begin{document}` and `\end{document}` delimit the body. Everything outside this pair is the *preamble* (covered in Section 6.3).
- The text between the delimiters is the visible content.
- `$e^{\mathrm{i}\pi} + 1 = 0$` is inline math mode: text between dollar signs is typeset as mathematics.

Table 6.1: Compiling a $\text{\LaTeX}$ document on each platform

Platform	Command
Windows (Command Prompt or Git Bash)	<code>pdflatex hello.tex</code>
macOS (Terminal)	<code>pdflatex hello.tex</code>
Linux	<code>pdflatex hello.tex</code> or <code>latexmk -pdf hello.tex</code>

Note that two compilation passes are often required for documents with cross-references or a table of contents; `latexmk` handles this automatically.

Running `pdflatex hello.tex` produces several files. The important one is `hello.pdf`, which can be opened in any PDF viewer. The `.aux`, `.log`, and `.out` files are auxiliary; they can be deleted safely.

6.2 A More Complete Example

Exercise L-1 produced the minimal viable document. This section introduces a more realistic document that demonstrates the features most commonly needed in academic writing: a structured hierarchy of sections, an automatically generated table of contents, in-text citations linked to a bibliography, and the full range of mathematical display modes. Every feature shown here is used in the documents produced by this program.

Information

Exercise L-2: A Structured Document with Math and Bibliography Create two files in the same folder. The first is the main document, `report.tex`. The second is the bibliography database, `sample.bib`. Both files must be in the same directory for the bibliography to resolve.

File 1: `report.tex`

```

1 \documentclass{article}
2
3 \usepackage{amsmath}      % extended math environments
4 \usepackage{booktabs}    % professional tables
5 \usepackage{hyperref}    % clickable links and
                           cross-references

```

```
6
7 \title{A Short Report on Mathematical Notation}
8 \author{Your Name}
9 \date{\today}
10
11 \begin{document}
12
13 \maketitle
14 \tableofcontents
15 \newpage
16
17 \section{Introduction}
18
19 This report demonstrates document structure and
20 mathematical
21 notation in \LaTeX{}. For background on literate
22 programming,
23 see Knuth~\cite{knuth1984literate}. The \LaTeX{} system
24 itself
25 is described in~\cite{lamport1994latex}.
26
27 \subsection{Motivation}
28
29 Scientific writing requires precise notation. \LaTeX{}
30 provides
31 that precision through its mathematical typesetting
32 engine.
33
34 \subsubsection{A Note on Style}
35
36 Use display equations for any expression that warrants
37 visual
38 emphasis, and inline math for symbols embedded in prose.
39
40 \section{Mathematical Display Modes}
41
42 \subsection{Inline Math}
43
44 Inline math is enclosed in single dollar signs:  $f(x) =$ 
45  $x^2 + 1$ .
46 It flows within the surrounding text without a line
47 break.
48
49 \subsection{Display Math (unnumbered)}
50
51 The preferred form for unnumbered display equations is
52 the
```

```

44 \texttt{\textbackslash[...] environment:
45
46 \[
47     \int_{-\infty}^{\infty} e^{-x^2}\,dx = \sqrt{\pi}
48 \]
49
50 The legacy \texttt{\$ \$... \$ \$} form produces the same
    visual
51 result but is deprecated in \LaTeX{} and should be
    avoided in
52 new documents.
53
54 \subsection{Numbered Equations}
55
56 The \texttt{equation} environment produces a numbered
    equation
57 that can be cross-referenced:
58
59 \begin{equation}
60     \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i
61     \label{eq:mean}
62 \end{equation}
63
64 Equation~\ref{eq:mean} defines the sample mean. An
    approximation
65 for the standard deviation is given
    in~\cite{gurland1971simple}.
66
67 \subsection{Multi-line Aligned Equations}
68
69 The \texttt{align} environment aligns multiple
    equations at a
70 designated column, marked with \texttt{\&}:
71
72 \begin{align}
73     \mu      &= \mathbb{E}[X] & \label{eq:mu} \\
74     & \\
75     \sigma^2 &= \mathbb{E}[(X-\mu)^2] & \label{eq:var}
76 \end{align}
77
78 Equations~\ref{eq:mu} and~\ref{eq:var} define the mean
    and
79 variance of a random variable  $X$ .
80
81 \section{A Simple Table}
82 \begin{table}[htbp]

```

```

83 \centering
84 \caption{Comparison of equation display modes}
85 \label{tab:modes}
86 \begin{tabular}{lll}
87 \toprule
88 Mode & Syntax & Use case \\
89 \midrule
90 Inline &  $\& \texttt{\textbackslash textttt\{\$...\$}}$  & &
      Symbols in prose \\
91 Unnumbered &  $\& \texttt{\textbackslash textttt\{\textbackslash textbackslash [...\textbackslash textbackslash]}}$  & &
      & Emphasis, no reference needed \\
92 Numbered &  $\& \texttt{\textbackslash textttt\{equation}}$  & &
      Cross-referenceable \\
93 Aligned &  $\& \texttt{\textbackslash textttt\{align}}$  & &
      Multi-line derivations \\
94 \bottomrule
95 \end{tabular}
96 \end{table}
97
98 \bibliographystyle{plain}
99 \bibliography{sample}
100
101 \end{document}

```

File 2: sample.bib

```

1 @article{knuth1984literate,
2   author   = {Knuth, Donald E.},
3   title    = {Literate Programming},
4   journal  = {The Computer Journal},
5   year     = {1984},
6   volume   = {27},
7   number   = {2},
8   pages    = {97--111}
9 }
10
11 @book{lamport1994latex,
12   author    = {Lamport, Leslie},
13   title     = {\LaTeX: A Document Preparation System},
14   edition   = {2nd},
15   publisher = {Addison-Wesley},
16   year      = {1994}
17 }
18
19 @article{gurland1971simple,
20   author = {Gurland, John and Tripathi, Ram C.},
21   title  = {A Simple Approximation for Unbiased

```

```

      Estimation
22         of the Standard Deviation},
23   journal = {The American Statistician},
24   year    = {1971},
25   volume  = {25},
26   number  = {4},
27   pages   = {30--32}
28 }

```

Warning

Compiling with a bibliography requires four steps. A single `pdflatex` pass is not sufficient when the document contains `\cite{}` commands. The full sequence is:

```

pdflatex report.tex
bibtex report
pdflatex report.tex
pdflatex report.tex

```

The first pass creates the `.aux` file listing all citation keys. `bibtex` reads the `.aux` file and the `.bib` database to produce a `.bbl` file containing the formatted bibliography. The second and third `pdflatex` passes incorporate the bibliography and resolve all cross-references. Running `latexmk -pdf report.tex` performs all necessary passes automatically.

Table 6.2: Equation display modes in $\text{\LaTeX}$

Mode	Syntax	Output style	Notes
Inline	<code>\$...\$</code>	Within text, compact	Use for symbols and short expressions in prose
Display (legacy)	<code>\$\$...\$\$</code>	Centered, full size	Deprecated; avoid in new documents
Display (preferred)	<code>\[...\]</code>	Centered, full size	Use for unnumbered display equations
Numbered	<code>\begin{equation} ... \end{equation}</code>	Centered with number	Cross-referenceable via <code>\label/\ref</code>
Multi-line aligned	<code>\begin{align} ... \end{align}</code>	Multiple lines, aligned at &	Each line can carry its own <code>\label</code>

6.3 Document Structure

Every $\text{\LaTeX}$ source file has two parts: the *preamble* (everything before `\begin{document}`) and the *body* (everything between `\begin{document}` and `\end{document}`). The preamble declares the document class, loads packages with `\usepackage{...}`, and sets metadata. The body contains the visible content, organized with sectioning commands.

```

1 \documentclass{article}
2
3 % --- Preamble ---
4 \usepackage{amsmath}      % extended math
5 \usepackage{graphicx}     % figures
6 \usepackage{booktabs}    % professional tables
7 \usepackage{hyperref}    % clickable links
8
9 \title{My First Report}
10 \author{Alice Smith}
11 \date{\today}
12
13 % --- Body ---
14 \begin{document}
15
16 \maketitle
17
18 \section{Introduction}
19 This is the introduction.
20
21 \subsection{Background}
22 Some background material.
23
24 \subsubsection{Details}
25 More specific details.
26
27 \section{Methods}
28 Description of methods.
29
30 \end{document}

```

Listing 6.1: Minimal compilable document with preamble and sectioning

The `\usepackage{...}` command loads extension packages. The packages most relevant to this program are `amsmath` (extended math), `graphicx` (figures), `booktabs` (tables), and `hyperref` (clickable links).

6.4 Mathematics in $\text{\LaTeX}$: A Cheat Sheet

$\text{\LaTeX}$ distinguishes two math modes. Inline mode (`$...$`) typesets mathematics within a line of text. Display mode (`\[...\]` or the `equation` environment) centers

mathematics on its own line and assigns an equation number when using `equation`. Use display mode for any expression that warrants its own visual emphasis, and inline mode for symbols embedded in prose.

6.4.1 Common Mathematical Notation

Table 6.3: Common $\text{\LaTeX}$ mathematical notation

Category	$\text{\LaTeX}$ source	Rendered	Notes
Inline and display modes			
Inline math	<code>\$x + y = z\$</code>	$x + y = z$	Within a sentence
Display math	<code>\[x + y = z \]</code>	(centered block)	No number
Numbered equation	<code>\begin{equation}...\end{equation}</code>	(centered block)	Cross-referenceable
Basic arithmetic			
Addition	<code>a + b</code>	$a + b$	
Subtraction	<code>a - b</code>	$a - b$	
Multiplication (dot)	<code>a \cdot b</code>	$a \cdot b$	Preferred in math
Multiplication (cross)	<code>a \times b</code>	$a \times b$	Vectors, cross product
Division	<code>\frac{a}{b}</code>	$\frac{a}{b}$	Always use <code>\frac</code> in display
Superscript	<code>x^{2}</code>	x^2	Braces required for >1 char
Subscript	<code>x_{i}</code>	x_i	Braces required for >1 char
Both	<code>x_{i}^{2}</code>	x_i^2	
Common functions			
Square root	<code>\sqrt{x}</code>	$\sqrt{x}$	
n th root	<code>\sqrt[n]{x}</code>	$\sqrt[n]{x}$	
Exponential	<code>e^{x}</code> or <code>\exp(x)</code>	e^x	<code>\exp</code> preferred in display
Natural log	<code>\ln x</code>	$\ln x$	
Log base b	<code>\log_{b} x</code>	$\log_b x$	
Sine	<code>\sin x</code>	$\sin x$	Use <code>\sin</code> , not <code>sin</code>
Cosine	<code>\cos x</code>	$\cos x$	
Absolute value	<code> x </code> or <code>\lvert x \rvert</code>	$ x $	<code>\lvert</code> scales with content
Floor	<code>\lfloor x \rfloor</code>	$\lfloor x \rfloor$	
Ceiling	<code>\lceil x \rceil</code>	$\lceil x \rceil$	

Continued on next page

Category	L ^A T _E X source	Rendered	Notes
Summation and integration			
Sum	<code>\sum_{i=1}^n x_i</code>	$\sum_{i=1}^n x_i$	Limits above/below in display
Product	<code>\prod_{i=1}^n x_i</code>	$\prod_{i=1}^n x_i$	
Integral	<code>\int_a^b f(x)\,dx</code>	$\int_a^b f(x) dx$	<code>\</code> , adds thin space before dx
Limit	<code>\lim_{x \to 0} f(x)</code>	$\lim_{x \rightarrow 0} f(x)$	
Greek letters (common in biostatistics and ML)			
alpha	<code>\alpha</code>	α	Significance level, learning rate
beta	<code>\beta</code>	β	Regression coefficients
gamma	<code>\gamma</code> , <code>\Gamma</code>	γ , Γ	Gamma distribution
delta	<code>\delta</code> , <code>\Delta</code>	δ , Δ	Change, Dirac delta
epsilon	<code>\epsilon</code> , <code>\varepsilon</code>	ϵ , ε	Small quantity
theta	<code>\theta</code> , <code>\Theta</code>	θ , Θ	Parameters, angle
lambda	<code>\lambda</code> , <code>\Lambda</code>	λ , Λ	Poisson rate, eigenvalue
mu	<code>\mu</code>	μ	Mean
nu	<code>\nu</code>	ν	Degrees of freedom
pi	<code>\pi</code> , <code>\Pi</code>	π , Π	3.14..., product
rho	<code>\rho</code>	ρ	Correlation
sigma	<code>\sigma</code> , <code>\Sigma</code>	σ , Σ	Std dev, summation
tau	<code>\tau</code>	τ	Time constant
phi	<code>\phi</code> , <code>\Phi</code>	ϕ , Φ	Normal CDF, angle
chi	<code>\chi</code>	χ	Chi-squared test
omega	<code>\omega</code> , <code>\Omega</code>	ω , Ω	Frequency, sample space
Relations and logic			
Not equal	<code>\neq</code>	$\neq$	
Less/greater equal	<code>\leq</code> , <code>\geq</code>	$\leq$, $\geq$	
Approximately	<code>\approx</code>	$\approx$	
Proportional to	<code>\propto</code>	$\propto$	
Distributed as	<code>\sim</code>	$\sim$	
For all	<code>\forall</code>	$\forall$	
There exists	<code>\exists</code>	$\exists$	

Continued on next page

Category	L ^A T _E X source	Rendered	Notes
Implies	<code>\Rightarrow</code>	$\Rightarrow$	
If and only if	<code>\Leftrightarrow</code>	$\Leftrightarrow$	
Element of	<code>\in</code>	$\in$	
Subset	<code>\subset</code> , <code>\subseteq</code>	$\subset$, $\subseteq$	
Union, intersection	<code>\cup</code> , <code>\cap</code>	$\cup$, $\cap$	
Number sets			
Real numbers	<code>\mathbb{R}</code>	$\mathbb{R}$	Requires <code>amsfonts</code> or <code>amssymb</code>
Natural numbers	<code>\mathbb{N}</code>	$\mathbb{N}$	
Integers	<code>\mathbb{Z}</code>	$\mathbb{Z}$	
Rationals	<code>\mathbb{Q}</code>	$\mathbb{Q}$	
Complex	<code>\mathbb{C}</code>	$\mathbb{C}$	
Vectors and matrices			
Bold vector	<code>\mathbf{x}</code>	$\mathbf{x}$	Preferred in statistics
Vector with arrow	<code>\vec{x}</code>	$\vec{x}$	Physics convention
Transpose	<code>A^{\top}</code> or <code>A^T</code>	$A^{\top}$	
2×2 matrix	<code>\begin{bmatrix} a & b \\ c & d \end{bmatrix}</code>	$\begin{bmatrix} a & b \\ c & d \end{bmatrix}$	Requires <code>amsmath</code>
Norm	<code>\ x\ </code>	$\ x\ $	
Inner product	<code>\langle x, y \rangle</code>	$\langle x, y \rangle$	

6.5 Figures

Figures are included using the `figure` float environment and the `\includegraphics` command from the `graphicx` package. The `pdflatex` compiler accepts PDF, PNG, and JPG image formats.

```

1 \begin{figure}[htbp]
2   \centering
3   \includegraphics[width=0.7\textwidth]{images/my-plot.png}
4   \caption{Description of the figure.}
5   \label{fig:my-plot}
6 \end{figure}
```

Listing 6.2: Including a figure in L^AT_EX

The `[htbp]` option suggests placement preferences to L^AT_EX: here, top, bottom, or a dedicated page. The `\caption` and `\label` commands provide a numbered caption and a cross-reference target, respectively.

6.6 Tables

Tables are built with the `table` float and the `tabular` environment. The `booktabs` package provides clean horizontal rules. Vertical rules (`|`) are omitted by convention in professional documents; `booktabs` horizontal rules are sufficient.

```

1 \begin{table}[htbp]
2 \centering
3 \caption{Sample measurements}
4 \label{tab:sample}
5 \begin{tabular}{lrr}
6 \toprule
7 Subject & Height (cm) & Weight (kg) \\
8 \midrule
9 Alice & 165 & 60 \\
10 Bob & 178 & 75 \\
11 Carol & 172 & 68 \\
12 \bottomrule
13 \end{tabular}
14 \end{table}

```

Listing 6.3: A minimal table with `booktabs`

6.7 Bibliography

L^AT_EX manages bibliographies via an external file (`.bib`) and a style command. The `\cite{key}` command inserts a citation at any point in the text; the `\bibliography{filename}` and `\bibliographystyle{plain}` commands generate the reference list.

For more sophisticated bibliography management, the `biblatex` package with the `biber` backend is preferred in modern documents. This repository’s shared preamble has bibliography support commented out; students writing independent reports should uncomment and configure it.

6.8 LaTeX Workshop in Visual Studio Code

The LaTeX Workshop extension for VSC (referenced in Section 1.4) provides a complete integrated workflow: syntax highlighting, live PDF preview, one-click build, and error navigation. Once installed, opening any `.tex` file activates the extension automatically. The build shortcut is `Ctrl+Alt+B` (Windows/Linux) or `Cmd+Option+B` (macOS), which runs `pdflatex` and displays the result in the built-in PDF viewer. Students do not need to use the command line for compilation after this is set up; the command-line approach in Section 6.1 is presented first so the underlying process is understood.

6.9 Summary

This chapter introduced $\text{\LaTeX}$, the document preparation system used throughout this program and widely adopted as the standard in science and mathematics:

- **What $\text{\LaTeX}$ is.** A plain-text markup language compiled into typeset PDFs, in contrast to WYSIWYG word processors.
- **The source-to-PDF workflow.** Writing a `.tex` file in a text editor, compiling with `pdflatex` or `latexmk`, and viewing the resulting PDF.
- **Document structure.** The preamble (document class, packages, meta-data) and the body (content organized with `\section`, `\subsection`, and `\subsubsection`).
- **Inline and display math.** Dollar signs for inline mathematics and `\[...\]` or the `equation` environment for display mathematics.
- **Equation modes.** Five modes exist: inline (`$...$`), legacy display (`$$...$$`, deprecated), preferred unnumbered display (`\[...\]`), numbered (`equation`), and multi-line aligned (`align`). Use `\[...\]` for unnumbered and `equation` for cross-referenceable display equations.
- **The math cheat sheet.** A comprehensive reference table (Table 6.3) covering arithmetic, functions, summation, integration, Greek letters, relations, number sets, and vectors/matrices.
- **Figures and tables.** Float environments with `\includegraphics`, `tabular`, and `booktabs` for professional formatting.
- **Bibliography workflow.** A `.bib` file stores reference entries. The `\cite{key}` command inserts a citation; `\bibliography{filename}` and `\bibliographystyle{plain}` generate the reference list. Documents with citations require a four-step compilation sequence: two `pdflatex` passes enclosing one `bibtex` pass, followed by a final `pdflatex`. `latexmk` handles this automatically.
- **LaTeX Workshop.** The VSC extension that provides syntax highlighting, live preview, and one-click build, eliminating the need for command-line compilation in daily use.

Chapter 7

Version Control with Git and GitHub

7.1 Configuring SSH Authentication for GitHub on Linux

GitHub no longer supports password authentication for Git operations over HTTPS. Instead, authentication must be performed using either a Personal Access Token or an SSH key. The recommended method for Linux environments is SSH authentication.

The following procedure generates an SSH key and configures Git to authenticate with GitHub.

7.1.1 Generate an SSH Key

Create a new SSH key using the `ed25519` algorithm:

```
1 ssh-keygen -t ed25519 -C "your_email@example.com"
```

When prompted for the file location, press **Enter** to accept the default:

```
1 ~/.ssh/id_ed25519
```

Optionally provide a passphrase for additional security.

7.1.2 Start the SSH Agent

Start the SSH agent and add the newly created key:

```
1 eval "$(ssh-agent -s)"
2 ssh-add ~/.ssh/id_ed25519
```

7.1.3 Add the Public Key to GitHub

Display the public key:

```
1 cat ~/.ssh/id_ed25519.pub
```

Copy the entire output and add it to GitHub:

1. Navigate to GitHub → Settings → SSH and GPG keys
2. Click New SSH key
3. Paste the copied key and save

7.1.4 Verify the Connection

Test the authentication:

```
1 ssh -T git@github.com
```

A successful configuration will display a message similar to:

```
1 Hi <username>! You've successfully authenticated.
```

7.1.5 Clone a Repository Using SSH

Repositories must be cloned using the SSH URL:

```
1 git clone git@github.com:USER/REPOSITORY.git
```

Using SSH avoids credential prompts and enables secure automated Git operations.

7.2 Configuring SSH Authentication for GitHub on Windows

Git supports two protocols for communicating with remote repositories: HTTPS and SSH. While HTTPS is simpler to set up initially, SSH authentication offers a more seamless workflow for frequent contributors because it eliminates repeated credential prompts. This section walks through the complete process of configuring SSH-based authentication for GitHub on a Windows machine using Git Bash.

7.2.1 Opening Git Bash

Git for Windows ships with its own bundled OpenSSH client and a Unix-like shell called Git Bash. All SSH commands in this section must be run inside Git Bash, not in Command Prompt or PowerShell, because the bundled `ssh`, `ssh-keygen`, and `ssh-agent` executables are located in the Git for Windows installation directory and are not automatically available in other terminals.

To open Git Bash, right-click on the Desktop or any folder in File Explorer and select **Git Bash Here** from the context menu. Alternatively, search for **Git Bash** in the Windows Start menu and launch it from there.

7.2.2 Generating an SSH Key

SSH authentication relies on a public/private key pair. The private key remains on your machine; the public key is uploaded to GitHub. Generate a new key pair using the `ed25519` algorithm, which is modern, compact, and recommended over the older RSA default:

```
1 ssh-keygen -t ed25519 -C "your_email@example.com"
```

Listing 7.1: Generate an ed25519 SSH key pair

Replace `your_email@example.com` with the address associated with your GitHub account. When prompted for a file location, press **Enter** to accept the default path:

```
1 C:\Users\YourName\.ssh\id_ed25519
```

Listing 7.2: Default key location on Windows

You will then be asked to enter an optional passphrase. A passphrase adds a second layer of protection in case your private key file is ever exposed; however, it must be entered each time the key is used (unless you configure the SSH agent to cache it). Press **Enter** twice to skip the passphrase if you prefer not to set one.

7.2.3 Starting the SSH Agent

The SSH agent is a background process that holds decrypted private keys in memory, so you do not have to re-enter a passphrase for every Git operation. On Linux and macOS the agent is typically started automatically by the desktop session, but on Windows the `ssh-agent` bundled with Git Bash must be started manually in each new terminal session.

Start the agent and add your private key with the following two commands:

```
1 eval "$(ssh-agent -s)"
2 ssh-add ~/.ssh/id_ed25519
```

Listing 7.3: Start the SSH agent and add the private key

The first command launches the agent and sets the necessary environment variables in the current shell. The second command registers your private key with the running agent. If you set a passphrase during key generation, you will be prompted to enter it now.

Tip

Automating Agent Startup The commands above must be repeated every time you open a new Git Bash session. To avoid this, you can add the following lines to your `~/.bashrc` file so that the agent starts automatically:

```
1 eval "$(ssh-agent -s)" > /dev/null 2>&1
2 ssh-add ~/.ssh/id_ed25519 2>/dev/null
```

Open or create `~/.bashrc` in a text editor, paste the two lines at the end of

the file, and save. The redirects suppress startup messages so they do not clutter your terminal. The next time you open Git Bash the agent will start and load your key without any manual steps.

7.2.4 Adding the Public Key to GitHub

GitHub needs your *public* key to verify your identity. Display its contents with:

```
1 cat ~/.ssh/id_ed25519.pub
```

Listing 7.4: Display the public key

The output will be a single long line beginning with `ssh-ed25519`. Select and copy the entire line, including the email address at the end.

Next, navigate to GitHub in your browser and follow these steps:

1. Click your profile photo in the upper-right corner and select **Settings**.
2. In the left sidebar, click **SSH and GPG keys**.
3. Click **New SSH key**.
4. Give the key a descriptive title (for example, *Windows laptop*).
5. Paste the copied public key into the **Key** field.
6. Click **Add SSH key** and confirm with your GitHub password if prompted.

7.2.5 Verifying the Connection

Before using SSH for any repository operations, confirm that GitHub can authenticate your key:

```
1 ssh -T git@github.com
```

Listing 7.5: Test the SSH connection to GitHub

On the first connection, SSH will display a fingerprint and ask whether you trust the host. Type **yes** and press **Enter**. If authentication succeeds, you will see a message similar to:

```
1 Hi username! You've successfully authenticated, but GitHub
  does
2 not provide shell access.
```

Listing 7.6: Expected response from GitHub

If instead you receive a **Permission denied** error, verify that the agent is running (`ssh-add -l` should list your key) and that the correct public key has been added to your GitHub account.

7.2.6 Cloning a Repository Using SSH

Once authentication is confirmed, clone repositories using the SSH remote URL:

```
1 git clone git@github.com:USER/REPOSITORY.git
```

Listing 7.7: Clone a repository over SSH

Replace `USER` with the repository owner's GitHub username and `REPOSITORY` with the repository name.

Warning

Use the SSH Remote URL, Not HTTPS The SSH remote URL begins with `git@github.com:` and ends with `.git`. It is *not* the same as the HTTPS URL, which begins with `https://github.com/`. If you clone or push using the HTTPS form after configuring SSH authentication, Git will still prompt you for a GitHub username and password (or personal access token). Always verify the remote URL with `git remote -v` and update it if necessary:

```
1 git remote set-url origin
   → git@github.com:USER/REPOSITORY.git
```

Table 7.1: SSH setup comparison: Linux/macOS vs. Windows (Git Bash).

Step	Linux / macOS	Windows (Git Bash)
Open terminal	Open the system terminal application (Terminal, iTerm2, etc.)	Right-click Desktop or folder and select <i>Git Bash Here</i> , or search Git Bash in the Start menu
Generate key	<code>ssh-keygen -t ed25519 -C "email"</code> (stored in <code>~/.ssh/</code>)	Same command in Git Bash; default path is <code>C:\Users\Name\.ssh\</code>
Start agent	Agent is typically started automatically by the desktop session; run <code>eval "\$(ssh-agent -s)"</code> if needed	Must be started manually each session: <code>eval "\$(ssh-agent -s)"</code>
Add key to agent	<code>ssh-add ~/.ssh/id_ed25519</code>	Same command in Git Bash
Display public key	<code>cat ~/.ssh/id_ed25519.pub</code>	Same command in Git Bash
Verify connection	<code>ssh -T git@github.com</code>	Same command in Git Bash
Clone with SSH	<code>git clone git@github.com:USER/REPO.git</code>	Same command in Git Bash

7.2.7 Pushing to Multiple Remote Repositories

A single local repository can maintain simultaneous connections to any number of remote repositories. This is useful when a project must be mirrored across an organisation account and a personal account (for example, keeping the DAIR3 workshop materials synchronised between the DAIR3 organisation and the personal `YourVeryOwnGitHubAccount` GitHub account).

7.2.7.1 Concepts

Git identifies remote repositories by short *names*. The name `origin` is the conventional default assigned when a repository is first cloned. Additional remotes can be registered under any name. Each remote entry stores a fetch URL and a push URL, both of which can be set independently.

Information

A remote name is purely local: it exists only in the `.git/config` file on your machine and is never pushed to any server. Two developers cloning the same repository can give the same remote entirely different names without consequence.

7.2.7.2 Listing Registered Remotes

Before adding a remote, confirm what is already registered:

```
1 git remote -v
```

Listing 7.8: List all remotes and their URLs

Each remote appears twice, once for fetch and once for push. A freshly cloned repository typically shows only `origin`:

```
1 origin https://github.com/DAIR3/DAIR3-Workshop.git (fetch)
2 origin https://github.com/DAIR3/DAIR3-Workshop.git (push)
```

7.2.7.3 Adding a Second Remote

Register an additional remote with `git remote add`:

```
1 git remote add YourVeryOwnGitHubAccount \
2 https://github.com/DAIR3/DAIR3-Workshop.git
```

Listing 7.9: Register a personal mirror remote (Linux / WSL)

```
1 git remote add YourVeryOwnGitHubAccount '
2 https://github.com/DAIR3/DAIR3-Workshop.git
```

Listing 7.10: Register a personal mirror remote (PowerShell)

Verify the result:

```
1 git remote -v
```

The output should now show four lines:

```
1 YourVeryOwnGitHubAccount
  https://github.com/YourVeryOwnGitHubAccount/DAIR3.git
  (fetch)
2 YourVeryOwnGitHubAccount
  https://github.com/YourVeryOwnGitHubAccount/DAIR3.git
  (push)
3 origin                    https://github.com/DAIR3/DAIR3-Workshop.git
  (fetch)
4 origin                    https://github.com/DAIR3/DAIR3-Workshop.git (push)
```

7.2.7.4 Pushing to Each Remote

With two remotes registered, every push must be issued explicitly to each destination. Git does not push to multiple remotes automatically unless a push target is configured:

```
1 git push origin HEAD
2 git push YourVeryOwnGitHubAccount HEAD
```

Listing 7.11: Push to both remotes explicitly

HEAD resolves to the current branch, which is the safest form to use in scripts because it does not depend on knowing the branch name in advance.

7.2.7.5 Checking Whether a Remote Exists Before Pushing

In automated scripts it is good practice to verify that a remote is registered before attempting to push to it, so a missing remote produces a clear diagnostic rather than a fatal error.

```
1 if git remote get-url YourVeryOwnGitHubAccount > /dev/null
  ↪ 2>&1; then
2     git push YourVeryOwnGitHubAccount HEAD
3 else
4     echo "Remote 'YourVeryOwnGitHubAccount' not configured."
5     echo "Register it with:"
6     echo "  git remote add YourVeryOwnGitHubAccount \
7 https://github.com/YourVeryOwnGitHubAccount/DAIR3.git"
8 fi
```

Listing 7.12: Guard a push with a remote existence check (bash)

```
1 git remote get-url YourVeryOwnGitHubAccount >nul 2>&1
2 if errorlevel 1 (
3     echo Remote 'YourVeryOwnGitHubAccount' not configured.
4     echo Register it with:
```

```

5     echo    git remote add YourVeryOwnGitHubAccount ^
6             https://github.com/YourVeryOwnGitHubAccount/DAIR3.git
7 ) else (
8     git push YourVeryOwnGitHubAccount HEAD
9 )

```

Listing 7.13: Guard a push with a remote existence check (PowerShell / batch)

7.2.7.6 Managing Remotes

Table 7.2 summarises the full set of remote management commands.

Table 7.2: Git remote management commands

Command	Effect
<code>git remote -v</code>	List all remotes with their fetch and push URLs.
<code>git remote add <n> <url></code>	Register a new remote under the given name.
<code>git remote remove <n></code>	Delete a remote entry from the local configuration.
<code>git remote rename <old> <new></code>	Rename a remote without changing its URL.
<code>git remote set-url <n> <url></code>	Update the URL of an existing remote.
<code>git remote get-url <n></code>	Print the URL of a named remote; exits with an error if the remote does not exist (useful in scripts).
<code>git remote show <n></code>	Display detailed information about a remote, including tracked branches and push configuration.

7.2.7.7 Alternative: a Single Remote with Multiple Push URLs

Git also supports configuring one remote name to push to two URLs simultaneously, so that a single `git push origin HEAD` reaches both destinations. This is convenient but reduces visibility; a failure on the second URL can be easy to miss in the output.

```

1 # The first push URL is already set from the clone.
2 # Add the personal repo as an additional push target:
3 git remote set-url --add --push origin \
4     https://github.com/DAIR3/DAIR3-Workshop.git
5 git remote set-url --add --push origin \
6     https://github.com/YourVeryOwnGitHubAccount/DAIR3.git

```

Listing 7.14: Add a second push URL to the origin remote

Warning

When using `set-url --add --push`, note that the *first* call replaces the default push URL rather than adding to it. Both URLs must be specified explicitly as shown above. Verify the result with `git remote -v` before relying on this configuration in automated scripts.

For the ALICE project, the explicit two-remote approach (separate `git push` calls per remote) is preferred because `gh.bat` checks `errorlevel` after each push and can report failures independently. A combined push URL would surface only the second failure.

7.3 Summary

This chapter introduced version control with Git and GitHub, focusing on the practical steps required to authenticate, clone, and synchronise repositories.

- **SSH key-based authentication.** GitHub no longer supports password authentication for Git operations. This chapter covered the complete SSH setup procedure on both Linux and Windows (using Git Bash), including key generation with the `ed25519` algorithm, adding the public key to GitHub, and verifying the connection.
- **Cloning repositories via SSH.** Once authentication is configured, repositories are cloned using the `git@github.com:` URL form, which avoids credential prompts on every push and pull.
- **Multiple remote repositories.** A single local repository can push to more than one remote. This chapter demonstrated how to register a second remote and push to both, keeping organisation and personal copies in sync.
- **Automated staging, committing, and pushing with `gh.bat`.** The `gh.bat` script automates the add-commit-push cycle, including version stamping and pushing to both remotes, reducing repetitive manual steps to a single command.

Chapter 8

Local LLM Infrastructure with Ollama

8.1 Introduction

For students: this chapter has two parts relevant to you. Section 8.7 explains how to configure the API keys required to run the agent programs in `resources/unit_7/F00/`. The Ollama CLI reference in Section 8.4 covers the commands you will use during the LLM unit exercises. You do not need to read the remainder of this chapter to complete the data challenge.

For instructors and program staff: this chapter documents the full configuration of the two-machine LLM serving infrastructure used by the DAIR3 program, covering the Dell Precision 7770 application host and the NVIDIA Spark inference server. It includes Ollama installation, `systemd` service configuration, network access control, memory budget management, single-machine and two-machine deployment modes, connectivity testing, and capacity planning.

8.2 Hardware Inventory

The two machines participating in this setup have fundamentally different memory architectures, which governs model selection and agent capacity.

8.2.1 Dell Precision 7770 (Application Host)

- **CPU RAM:** 128 GB (112 GB available after OS overhead)
- **GPU:** NVIDIA RTX A5500 Laptop GPU, 16 GB VRAM (discrete)
- **Architecture:** x86_64
- **OS:** Windows 11 with WSL2 Ubuntu 24.04

- **Role:** Runs a FastAPI application, PostgreSQL 16, and the agent orchestration layer. Hosts a local Ollama instance for single-machine development mode.

On the Dell, the RTX A5500 VRAM (16 GB) is *separate* from system RAM. Ollama loads model weights into VRAM first and spills to system RAM only when the model exceeds VRAM capacity. Inference degrades significantly once spilling begins: tokens per second can drop by an order of magnitude for spilled portions of the model.

8.2.2 NVIDIA Spark (LLM Server)

- **Unified memory:** 128 GB physical; approximately 120 GB visible to Ollama (110 GB conservatively available)
- **GPU:** NVIDIA GB10 SoC
- **Architecture:** ARM64 (AArch64)
- **OS:** Ubuntu 24.04 (native, no virtualisation)
- **Role:** Dedicated LLM inference server. CPU and GPU share the same memory pool, so the entire 110 GB is available for model weights and KV cache simultaneously.

Warning

The GB10 uses a *unified memory* architecture. Unlike a discrete GPU, there is no separate VRAM budget to track. The 110 GB pool covers weights, KV cache, and OS overhead together. This makes the Spark well suited to serving multiple large models simultaneously (provided context windows are bounded), but it also means that vLLM's PagedAttention, which is optimised for discrete high-VRAM GPUs, provides fewer benefits here than Ollama's native unified-memory handling.

8.3 Network Topology

Both machines reside on the UTSA `utsarr.net` subnet. At the time of configuration they held the following addresses:

Machine	Interface	IP Address	Subnet
Spark	enP7s7	10.2.14.24	/23
Dell	Ethernet 8	10.2.14.143	/23
Spark	enP7s7 MAC	4c:bb:47:2c:74:eb	
Dell	Ethernet 8 MAC	ac:91:a1:7d:06:ff	

8.3.1 Switch vs. Router

A Netgear ProSAFE GS105 unmanaged 5-port switch connects the two machines. An *unmanaged switch* operates at OSI Layer 2: it forwards Ethernet frames between ports based on MAC addresses but has no concept of IP subnets, routing, or traffic restriction. A *router* operates at Layer 3 and is required to separate subnets or enforce IP-level traffic policies.

Because both machines are on the same /23 subnet managed by the university network, ARP entries are not visible between hosts; traffic crosses at least one router hop, and ARP does not traverse routers. This is expected behaviour on a routed network and should not be interpreted as a connectivity error.

8.3.2 MAC Address Filtering

MAC-based firewall rules (via `iptables`) are only meaningful on the same Layer 2 segment. They can be spoofed trivially and should be treated as an informational layer rather than a security boundary. IP firewall rules (`ufw`) are the appropriate primary control for restricting Ollama access to known client addresses.

```
1 # On the Spark: allow only the Dell, deny all other sources
2 sudo ufw allow from 10.2.14.143 to any port 11434
3 sudo ufw deny 11434
```

Listing 8.1: Restrict Ollama to the Dell's IP only

8.3.3 Identifying Network Interfaces

Use the following commands to enumerate interfaces and confirm addresses before configuring firewall rules.

```
1 ip a
2 # Look for interfaces with state UP and a valid inet
   ↪ address.
3 # The link/ether line on each interface gives the MAC
   ↪ address.
```

Listing 8.2: Linux: list interfaces and addresses

```
1 ipconfig /all
2 # Look for active Ethernet adapters with an IPv4 Address.
3 # The Physical Address field gives the MAC (dash-separated
   ↪ format).
```

Listing 8.3: Windows: list interfaces and addresses

Information

MAC address formats. Linux tools (`ip a`, `arp`) report MACs in lowercase colon format: `ac:91:a1:7d:06:ff`. Windows `ipconfig /all` reports them in uppercase dash format: `AC-91-A1-7D-06-FF`. Both represent the same

address. Configuration files and scripts should normalise to a single format before comparison.

8.4 Installing Ollama

8.4.1 Linux (Spark)

The official installer script detects the architecture automatically and selects the appropriate binary (ARM64 in this case).

```
1 curl -fsSL https://ollama.com/install.sh | sh
```

Listing 8.4: Install Ollama on Ubuntu

The installer creates the `ollama` user and group, adds the current user to the `ollama` group, installs the binary to `/usr/local/bin/ollama`, and registers a `systemd` service. Confirm the installation:

```
1 ollama --version
2 systemctl status ollama
```

8.4.2 Windows (Dell)

Download the Windows installer from <https://ollama.com> and run it. Ollama registers itself in the system tray and starts automatically. The API listens on `127.0.0.1:11434` by default.

8.4.3 Conda Environments

The Spark ships with Miniconda, which activates a `(base)` environment by default. Ollama does not require Python and should be installed at the system level, not inside a Conda environment. Disable automatic Conda activation to avoid interference with system tooling:

```
1 conda config --set auto_activate_base false
```

For all Python work in this stack, use plain `venv` environments, which translate directly to `systemd` `ExecStart` paths and maintain production parity without Conda's dependency overhead.

8.5 Configuring Ollama for Network Access

By default Ollama binds only to `localhost`. For the two-machine setup the Spark must expose its API to the network. Configure this through a `systemd` override so the setting survives service restarts and reboots.


```
1 sudo systemctl edit ollama
```

Listing 8.5: Create a systemd override on the Spark

This opens `/etc/systemd/system/ollama.service.d/override.conf`. Add the following:

```
1 [Service]
2 Environment="OLLAMA_HOST=0.0.0.0:11434"
3 Environment="OLLAMA_MAX_LOADED_MODELS=4"
4 Environment="OLLAMA_NUM_PARALLEL=4"
5 Environment="OLLAMA_KEEP_ALIVE=24h"
6 Environment="OLLAMA_NUM_CTX=8192"
```

Listing 8.6: Spark Ollama systemd override

Apply the changes:

```
1 sudo systemctl daemon-reload
2 sudo systemctl restart ollama
3 systemctl status ollama
```

The environment variables are explained in Table 8.1.

Table 8.1: Key Ollama environment variables

Variable	Effect
OLLAMA_HOST	Bind address and port. Use <code>0.0.0.0:11434</code> to accept connections from any network interface.
OLLAMA_MAX_LOADED_MODELS	Maximum number of models resident in memory simultaneously. Prevents thrashing when MAX dispatches concurrent agent calls.
OLLAMA_NUM_PARALLEL	Maximum concurrent inference requests per model.
OLLAMA_KEEP_ALIVE	Duration a model remains loaded after its last request. The default is 5 minutes; <code>24h</code> keeps models warm across a workday.
OLLAMA_NUM_CTX	Default context window in tokens. Critical on the Spark: without this cap, Ollama defaults to 262,144 tokens, which allocates a KV cache large enough to exceed physical memory on most models. Set to 8192 as a safe default.

8.6 Memory Budget and Context Window Management

8.6.1 The Governing Formula

Total memory consumed per agent has two independent components:

$$M_{\text{agent}} = M_{\text{weights}} + M_{\text{KV}} \quad (8.1)$$

where the weight footprint under Q4 quantisation is approximated by:

$$M_{\text{weights}} \approx 0.5 \text{ GB} \times B_{\text{params}} \quad (8.2)$$

and the KV cache footprint is:

$$M_{\text{KV}} \approx \frac{n_{\text{layers}} \times n_{\text{kv_heads}} \times d_{\text{head}} \times C \times 2 \times 2}{10^9} \text{ GB} \quad (8.3)$$

where C is the context length in tokens and the factor of 2 appears twice for the key/value pair and the 2-byte (BF16) dtype.

Warning

The context window dominates memory for large models. A llama3.3:70b model requires approximately 40 GB for weights at Q4. With Ollama's default context of 262,144 tokens, the KV cache adds another 134 GB, pushing total allocation to 174 GB, exceeding the Spark's 120 GB and causing the load to fail. With a context of 8,192 tokens, the KV cache drops to 2.5 GB, giving a total of 42.5 GB which fits comfortably.

8.6.2 The Modelfile Pattern for Large Models

Ollama does not support runtime context overrides via the `--num-ctx` flag. The correct mechanism is a *Modelfile*, which creates a named derivative of the base model with the parameter baked in.

8.6.2.1 Linux (Spark and WSL)

```
1 cat > /tmp/Modelfile.llama70b << 'EOF'
2 FROM llama3.3:70b
3 PARAMETER num_ctx 8192
4 EOF
5 ollama create llama3.3:70b-ctx8k -f /tmp/Modelfile.llama70b
6 ollama run llama3.3:70b-ctx8k "Reply with one word: ready"
```

Listing 8.7: Create a context-bounded model variant on Linux

8.6.2.2 Windows (PowerShell)

PowerShell uses a here-string syntax for multi-line literals. The closing `@` must appear at the *absolute start of a line* with no leading spaces or tabs; this is a hard requirement of the PowerShell parser.

```
1 @"
2 FROM llama3.3:70b
3 PARAMETER num_ctx 8192
```

```

4 "@ | Out-File -FilePath "$env:TEMP\Modelfile.llama70b"
   ↪ -Encoding utf8
5 ollama create llama3.3:70b-ctx8k -f
   ↪ "$env:TEMP\Modelfile.llama70b"

```

Listing 8.8: Create a context-bounded model variant on Windows

Apply the same pattern to every model above 20 B parameters. The naming convention `<model>-ctx8k` makes the operational constraint explicit at the call site.

8.6.3 Hardware Agent Capacity

Given the memory budget formula, the maximum number of agents that can run simultaneously on a machine is:

$$N_{\text{hw}} = \left\lfloor \frac{M_{\text{available}}}{M_{\text{agent}}} \right\rfloor \quad (8.4)$$

Table 8.2 gives the confirmed model inventory with measured status on each platform and approximate memory footprint at 8,192-token context.

Table 8.2: Confirmed model inventory and memory footprint at 8K context

Model	Ollama name	Params (B)	M_w (GB)	M_{KV} (GB)	M_{total} (GB)	Status
nemotron-3-super:120b	...-ctx8k	120 (MoE)	65	4.0	69.0	✓ both
llama3.3:70b	...-ctx8k	70	40	2.5	42.5	✓ both
qwen3:32b	...-ctx8k	32	18	1.3	19.3	✓ both
mistral-small:22b	mistral-small:22b	22	13	1.0	14.0	✓ both
phi4:14b	phi4:14b	14	8	0.8	8.8	✓ both
qwen3:14b	qwen3:14b	14	8	0.8	8.8	✓ both
mistral-nemo:12b	mistral-nemo:12b	12	7	0.7	7.7	✓ both
mistral:7b	mistral:7b	7	4	0.5	4.5	✓ both
deepseek-r1:8b	deepseek-r1:8b	8	5	0.5	5.5	✓ both
gemma3:4b	gemma3:4b	4	2.5	0.3	2.8	✓ both
phi4-mini:3.8b	phi4-mini:3.8b	3.8	2.5	0.3	2.8	✓ both
qwen3:4b	qwen3:4b	4	2.5	0.3	2.8	✓ Dell; † Spark
gemma3:1b	gemma3:1b	1	0.8	0.1	0.9	✓ both
deepseek-v3.1:671b	—	671 (MoE)	404	8.0	412	× too large

† ARM64 crash on Spark (Ollama bug); works on x86_64 Dell.

8.6.4 Thinking Models and Token Budgets

Several models in the inventory implement chain-of-thought reasoning internally, emitting a *thinking block* before their final answer. These include **qwen3** (all sizes), **deepseek-r1**, and **nemotron-3-super**. When calling these models via the OpenAI-compatible endpoint with a small `max_tokens` budget, the budget may be exhausted inside the thinking block, producing either an HTTP 500 error or an empty reply.

The correct mitigation is *not* to suppress thinking via the `/no_think` system prompt, which can leave the model with nothing to say outside the block. Instead, set `max_tokens` generously (500 or more) and strip thinking tags from the response:

```

1 # DeepSeek-R1 style:
2 <think> ... </think>
3
4 # Qwen3 / Ollama native style:
5 Thinking...
6 ... reasoning text ...
7 ...done thinking.
```

Listing 8.9: Thinking tag patterns to strip from model output

8.7 Configuring API Keys

You will receive an API key for OpenAI and Anthropic during our first session. You can get your own API for free at Groq. You need to create three API key variables following the procedure described below: `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `GROQ_API_KEY`. Test the programs located at `resources/unit_7/F00/` in the DAIR3 repository: `agentGroq.py`, `agentGPT.py`, and `agentClaude.py`.

On Windows, execute the following three commands one at a time for each API key. The example for `OPENAI_API_KEY` is as follows:

```

1 setx OPENAI_API_KEY "sk-... API key"
2 Exit
3 echo %OPENAI_API_KEY%
```

In the code above, replace `[sk-... API key]` with your actual API key. The first command sets the environment variable permanently for your user account. The second command closes the Command Prompt to ensure the new environment variable is registered. The third command, run in a new Command Prompt window, prints the stored value to verify it was set correctly.

On Mac/Linux execute the following three commands one at a time for each API key. The example for `OPENAI_API_KEY` is as follows:

```

1 echo "export OPENAI_API_KEY='sk-... API key'" >> ~/.zshrc
2 source ~/.zshrc
3 echo $OPENAI_API_KEY
```

In the code above, replace `'sk-... API key'` with your actual API key.

As reported by Bryan Fowler, if you experience errors trying to add your environmental variables in a Mac/Linux, the most likely problem is ownership of the `~/.zshrc` file. In this case, execute

```

1 sudo chown $(whoami) ~/.zshrc
```

And try to create the environmental variables again.

As reported by Drew Stephen regarding Mac, “*apparently it doesn’t work on the default c shell but switching to the zsh lets it run.*” You might need to close and reopen the terminal window from where you started for changes to become effective.

8.8 The FOO Framework and Agent Feasibility

8.8.1 Minimum Agent Count

The MAX Multi-Agent Consensus System implements the FOO algorithm for Byzantine-fault-tolerant consensus. Given that any individual agent has an invalidation probability p , the minimum number of agents required to achieve consensus confidence C is:

$$N_{\text{foo}}(p, C) = \left\lceil \frac{\log(1 - C)}{\log(p)} \right\rceil \quad (8.5)$$

Selected values are given in Table 8.3.

Table 8.3: Minimum agent count N_{foo} by invalidation rate and confidence target

Per-agent invalidation rate p	Confidence C	N_{foo}
0.30	0.95	4
0.30	0.99	6
0.20	0.95	3
0.20	0.99	5
0.10	0.99	3
0.05	0.99	2

8.8.2 Agent Feasibility Score

The *Agent Feasibility Score* (AFS) couples the hardware budget to the FOO validity condition:

$$\text{AFS} = \frac{N_{\text{hw}}}{N_{\text{foo}}} \quad (8.6)$$

A configuration is *feasible* if and only if $\text{AFS} \geq 1.0$. When $\text{AFS} < 1.0$, one or more of the following mitigations must be applied: reduce the context window (increases N_{hw}), use a smaller model (increases N_{hw}), or relax the confidence target C (reduces N_{foo}).

8.8.3 Reference Configurations

Dell VRAM-optimal (fast, no RAM spill). Five agents fitting entirely within 16 GB VRAM, yielding $\text{AFS} = 8.00$ at $p = 0.20$, $C = 0.99$:

Model	M_{agent} (GB)	Location
gemma3:1b	0.9	VRAM
phi4-mini:3.8b	2.8	VRAM
gemma3:4b	2.8	VRAM
qwen3:4b	2.8	VRAM
mistral:7b	4.5	VRAM
Total	13.8	(of 16 GB)

Spark full-capability configuration. Mixed-tier configuration using 110 GB unified memory:

Role	Model	M_{agent} (GB)	AFS
Orchestrator	qwen3:32b-ctx8k	19.3	1.67
Primary analyst	llama3.3:70b-ctx8k	42.5	0.67
Critic A	mistral-small:22b	14.0	2.33
Critic B	phi4:14b	8.8	4.00
Synthesiser	mistral-nemo:12b	7.7	4.67
Total		92.3	0.67

The minimum AFS of 0.67 (from llama3.3:70b alone in the pool) indicates that a full five-agent round cannot run simultaneously on the Spark with this model selection. The operational solution is to run the T1 models sequentially within a round and cache their outputs, or to replace llama3.3:70b-ctx8k with qwen3:32b-ctx8k to free memory for a fifth simultaneous agent.

8.9 Single-Machine Configuration (Dell)

In single-machine mode the Dell runs both the ALICE application stack and the Ollama inference layer. This configuration is appropriate for prompt development and unit-level agent testing where network isolation is desirable.

8.9.1 Environment Variable

```

1 LLM_PROVIDER=ollama
2 LLM_BASE_URL=http://127.0.0.1:11434/v1
3 LLM_MODEL=mistral:7b

```

Listing 8.10: .env for single-machine mode

From WSL, the Windows Ollama instance is reachable via the WSL default gateway rather than localhost:

```

1 ip route | grep default | awk '{print $3}'
2 # Typical result: 172.24.0.1 (varies by WSL version and
   ↪ config)

```

Listing 8.11: Find the Windows gateway IP from WSL

```

1 LLM_PROVIDER=ollama
2 LLM_BASE_URL=http://172.24.0.1:11434/v1
3 LLM_MODEL=mistral:7b

```

Listing 8.12: .env for WSL accessing Windows Ollama

8.9.2 Model Selection for Single-Machine Development

The Dell's 16 GB VRAM constrains simultaneous model capacity. For development work targeting the FOO framework, the following tier assignment balances capability with VRAM residency:

- **Primary agent:** `deepseek-r1:8b` (5.5 GB), reasoning model with thinking mode; good proxy for larger consensus agents.
- **Fast critic:** `mistral-nemo:12b` (7.7 GB), tools-capable, fast, NVIDIA-trained.
- **Utility agent:** `gemma3:1b` (0.9 GB), sub-second responses for structured output tasks.

Total VRAM: 14.1 GB out of 16 GB. This leaves 1.9 GB headroom for the display server and background processes.

8.10 Two-Machine Configuration

In the two-machine setup the Dell hosts the application stack and the Spark serves all LLM inference. The Spark's 110 GB unified memory pool allows multiple large models to remain resident simultaneously, which is the hardware prerequisite for running a full FOO consensus round without sequential eviction.

8.10.1 Architecture

+-----+ HTTP/REST +-----+	
Dell 7770	<----- NVIDIA Spark
	port 11434
Alice FastAPI	Ollama (port 11434)
PostgreSQL 16	systemd: ollama.service
Agents	OLLAMA_HOST=0.0.0.0
Python venv: ALICE	OLLAMA_NUM_CTX=8192
.env:	Models resident:
LLM_BASE_URL=	qwen3:32b-ctx8k
http://10.2.14.24:	llama3.3:70b-ctx8k
11434/v1	mistral-nemo:12b
+-----+	+-----+

8.10.2 Environment Variables

```

1 # Two-machine mode: Spark serves all inference
2 LLM_PROVIDER=ollama
3 LLM_BASE_URL=http://10.2.14.24:11434/v1
4 LLM_API_KEY=not-required
5
6 # Per-agent model assignment (Max routing policy)
7 AGENT_MODEL_ORCHESTRATOR=qwen3:32b-ctx8k
8 AGENT_MODEL_ANALYST=llama3.3:70b-ctx8k
9 AGENT_MODEL_CRITIC=mistral-nemo:12b
10 AGENT_MODEL_FAST=mistral:7b

```

Listing 8.13: .env for two-machine mode (Dell application host)

The `ProviderAdapter` protocol in MAX makes switching between single-machine and two-machine modes a one-line change to `LLM_BASE_URL`. No application code changes are required.

8.10.3 Connectivity Verification

A connectivity test script (`connectivity_test.py`) validates the end-to-end path from the application host to the Ollama server. It tests four conditions in sequence, stopping at the first failure:

1. **TCP handshake:** confirms the host is reachable and the port is open.
2. **MAC ARP check:** informational only; confirms whether the machines are on the same Layer 2 segment. Failure here is expected on a routed university network and does not indicate a problem.
3. **Model list:** confirms Ollama is serving and returns the installed model list.
4. **Inference:** sends a minimal chat completion request and validates the response.

Machine profiles, ports, and test parameters are stored in `connectivity_config.json`, which is excluded from version control (`.gitignore`) because it contains environment-specific IP and MAC addresses. A `connectivity_config.json.template` with placeholder values is committed alongside the script.

8.11 Capacity Testing

A capacity test script (`capacity_test.py`) reads the model registry from `connectivity_config.json`, computes the theoretical N_{hw} and AFS for each model, and empirically validates each installed model by running a single inference call.

8.11.1 Platform Considerations

- **ARM64 (Spark):** qwen3:4b fails with an `exit status 2` crash in the Ollama ARM64 binary. This is an Ollama bug affecting this specific model and quantisation on ARM64. The same model works correctly on the x86_64 Dell. The script detects architecture at runtime and skips ARM64-specific failures appropriately.
- **Modelfile variants:** Context-bounded variants (e.g. llama3.3:70b-ctx8k) must be created separately on each machine. A model created via Modelfile on the Spark is not automatically available on the Dell. The script detects *base installed but variant missing* and emits the exact shell commands to create the variant, using PowerShell syntax on Windows and bash heredoc syntax on Linux.
- **VRAM vs. RAM:** On the Dell, the script distinguishes models that fit entirely in VRAM (fast inference) from those that spill to system RAM (significantly slower). Two configuration recommendations are produced: a VRAM-optimal configuration (fastest, may not reach N_{foo}) and a RAM-extended configuration (reaches N_{foo} if RAM permits, but with a performance warning for spilled models).

8.11.2 Ollama CLI Reference

```

1 # List installed models
2 ollama list
3
4 # Show currently loaded models and memory usage
5 ollama ps
6
7 # Pull a model from the registry
8 ollama pull qwen3:32b
9
10 # Interactive chat session
11 ollama run mistral:7b
12
13 # One-shot prompt (no interactive session)
14 ollama run mistral:7b "Explain the rhizome model in one
    ↪ paragraph"
15
16 # Remove a model
17 ollama rm mistral:7b
18
19 # Test the OpenAI-compatible API directly
20 curl http://localhost:11434/v1/models
21 curl http://localhost:11434/v1/chat/completions \
22     -H "Content-Type: application/json" \
23     -d '{"model": "mistral:7b",

```

24

```
"messages":[{"role":"user","content":"Hello"}]}'
```

Listing 8.14: Essential Ollama CLI commands

8.12 Deployment Parity

A core principle of this infrastructure is that the development environment mirrors production without containerisation. Both machines run native Ubuntu (or WSL2 for the Windows host), Ollama is managed by `systemd`, and configuration is injected via environment files rather than code changes.

The two-tier configuration model is:

- **Development:** `.env` file in the project root, loaded by `python-dotenv` with `override=False`.
- **Production:** `systemd EnvironmentFile=` directive pointing to a file outside the repository. No code changes required at deployment.

Switching from single-machine to two-machine mode, or from the Dell's local Ollama to the Spark, requires only updating `LLM_BASE_URL` in the environment file. The `ProviderAdapter` protocol transparently routes requests to whichever endpoint is configured.

8.13 Setting Up Ollama with a GUI on Ubuntu

To transition from a command-line interface to a graphical experience, follow these steps to install the Ollama backend and the Open WebUI frontend.

8.13.1 Step 1: Install Ollama Backend

First, install the core Ollama service using the official installation script:

```
curl -fsSL https://ollama.com | sh
```

8.13.2 Step 2: Deploy Open WebUI via Docker

Open WebUI is the most popular ChatGPT-like interface for Ollama. Run the following Docker command to pull the image and start the container:

```
docker run -d -p 3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -v open-webui:/app/backend/data \
  --name open-webui \
  ghcr.io/open-webui/open-webui:main
```

8.13.3 Step 3: Access the Interface

Once the container is running, open your preferred web browser and navigate to:

`http://localhost:3000`

You will be prompted to create an admin account (stored locally) to begin chatting with your models.

8.13.4 Technical Considerations

- **GPU Support:** For NVIDIA users, ensure the `nvidia-container-toolkit` is installed to enable hardware acceleration within Docker.
- **External Access:** To access this GUI from another machine, set the environment variable `OLLAMA_HOST=0.0.0.0` in your `systemd` service configuration.

Chapter 9

2026 Data Challenge - Vital Statistics

9.1 Problem Description

Your team is part of a consultancy supporting a planning commission for the State of Texas. The commission is planning budgetary requirements for various State services in 2030. The commission requests the following:

- Projections of underweight newborns by county in Texas.
- Projections of newborn mortality by county in Texas. According to the CDC, a stillbirth is classified as either early, late, or term. An early stillbirth is a fetal death occurring between 20 and 27 completed weeks of pregnancy. A late stillbirth occurs between 28 and 36 completed pregnancy weeks. A term stillbirth occurs between 37 or more completed pregnancy weeks.
- Identification of the socioeconomic factors associated with these two outcomes.
- Comparison to other states.

Your team will produce an executive report accompanied by an appendix of technical material.

9.2 Data Description

The commission has approved specific data sources for this analysis. You can only use approved data available at this [hyperlink](#).

National Center for Health Statistics (NCHS): Since 1969, all births recorded in the US are available as digital records from the NCHS, from the Centers for Disease Control and Prevention (CDC). The date and time of birth is publicly available only between 1969 and 1988; starting in 1989, only the week of birth is recorded. The NCHS keeps records of place of birth, assistance during delivery (at home, with doctors, with midwives), level of education of the parents, place of

residence, weight at birth, number of weeks of gestation, number of siblings, birth order, etc. In total, over 100 variables are recorded. You have access to records from 1969 to 1988.

Surveillance, Epidemiology, and End Results Program (SEER): The U.S. Census Bureau annually releases unabridged population estimates for five-year age groups and race at the county level. The Census Bureau does not release bridged race estimates by single year of age at the county level due to concerns about the reliability of these estimates. However, these estimates are provided to the National Cancer Institute through SEER to meet programmatic needs such as the creation of age groupings that differ from the standard groupings used by the Census Bureau. Users of the single-year-of-age county-level interpolated race population estimates should carefully consider the limited reliability of these estimates. County-level population files with 19 age groups (<1, 1-4, ..., 80-84, 85+) and with 86 single-year age groups (<1, 1, 2, ..., 84, 85+) are provided.

Socioeconomic Data and Applications Center (SEDAC): SEDAC, the Socioeconomic Data and Applications Center, is one of the Distributed Active Archive Centers (DAACs) in the Earth Observing System Data and Information System (EOSDIS) of the U.S. National Aeronautics and Space Administration (NASA). SEDAC contains georeferenced U.S. county-level population projections, total and by sex, race and age, based on shared socioeconomic pathways (SSPs). This data set, produced by Mathew E. Hauer, consists of county-level population projection scenarios of population in five-year intervals for all U.S. counties for the period 2020 - 2100. Obtain the data description from <https://doi.org/10.1038/sdata.2019.5>.

The NCHS data set covers from 1969 through 1986; it provides individual birth data. The SEER data set provides age-bracketed population estimates from 1969 to 2020. Since for this exercise we do not have birth data beyond 1986, you will have to use the SEER data to infer births in the period 1987-2020; alternatively, you could go to the NCHS source and obtain more recent data, however the NCHS data is complex and downloading additional years is not advised in the short time available to complete the Rowdy Datathon (but we will not stop you). The SEDAC data has total population estimates per county from 2020 through 2100 categorized in four ethnicities: Hispanic, white, black, and other. A viable sequence of analysis is NCHS → SEER → SEDAC.

9.3 Identifying Underweight Cutoff Points

The CDC offers a data table of infant weight for age, available at:

<https://www.cdc.gov/growthcharts/data/zscore/wtageinf.xls>

You can extract from this table the information related to weight at birth.

The Excel file `wtageinf.xls` contains the LMS parameters and selected percentile values needed to compute exact percentiles and z -scores for anthropometric measurements. The parameters are provided by sex (1 = male, 2 = female) and by single month of age. Age is listed at the half-month point representing the entire month (e.g., 1.5 months corresponds to 1.0–1.99 months). The only exception is

birth, which represents the point at birth.

The LMS method summarizes the distribution of a measurement at a given age using three parameters:

- L : the Box–Cox power transformation,
- M : the median,
- S : the generalized coefficient of variation.

These parameters allow computation of both percentiles and z -scores.

To obtain the measurement value X corresponding to a given z -score Z (or percentile), use:

$$X = M(1 + LSZ)^{1/L}, \quad L \neq 0$$

$$X = M \exp(SZ), \quad L = 0$$

where L , M , and S are taken from the Excel row corresponding to the child's age (in months) and sex. The z -score corresponding to common percentiles is:

$$\begin{aligned} -1.881 &\leftrightarrow 3^{\text{rd}} \\ -1.645 &\leftrightarrow 5^{\text{th}} \\ -1.282 &\leftrightarrow 10^{\text{th}} \\ -0.674 &\leftrightarrow 25^{\text{th}} \\ 0 &\leftrightarrow 50^{\text{th}} \end{aligned}$$

Example: For a 9-month-old male, the WTAGEINF table gives:

$$L = -0.1600954, \quad M = 9.476500305, \quad S = 0.11218624.$$

Using $Z = -1.645$ (5th percentile), the cutoff is:

$$X = 7.90 \text{ kg.}$$

This value represents the 5th percentile weight-for-age and can be used as an underweight threshold if the 5th percentile definition is adopted.

Computing a Z-Score from a Measurement: To obtain the z -score corresponding to a given measurement X :

$$Z = \frac{(X/M)^L - 1}{LS}, \quad L \neq 0$$

$$Z = \frac{\ln(X/M)}{S}, \quad L = 0.$$

The corresponding percentile is then obtained from the standard normal distribution.

Example: For a 9-month-old male weighing 9.7 kg:

$$Z = 0.207,$$

which corresponds approximately to the 58th percentile.

Identifying Underweight Cutoff Points in Excel: To identify an underweight threshold in the Excel file:

1. Filter the data by sex (1 = male, 2 = female).
2. Locate the row corresponding to the child's age in months.
3. Identify the desired cutoff percentile (e.g., 5th or 3rd percentile column).
4. Alternatively, compute the exact cutoff using the LMS formula above.

A child is classified as underweight if their measured value falls below the selected percentile cutoff (e.g., below the 5th percentile or below $Z = -2$, depending on the chosen clinical definition).

If finer age resolution is required, linear interpolation between adjacent months may be applied to the L , M , and S parameters prior to computation.

9.4 Data Files

9.5 Criteria

Results will be evaluated according to requirements set by the commission:

- **Compelling presentation:** You must enable the commission to share your numerical and graphical results directly with legislators and citizens through executive summaries. This lay audience should find your summaries and implications to be understandable and convincing. Express your results in ways that can be acted on to plan e.g. funding of schools, care for the elderly, etc. The supporting documentation can be technical.
- **Analysis comprehension:** Before a single line of code is written, before a single byte of raw data is processed, you must be able to tell the story of what is the progression of steps that will be undertaken in analysis.
- **Sound technical methods:** You may cite the analyses of others, but the commission wants to see the methods that you have invented or adopted to calculate these projections (which should be accompanied by error bars, if possible). The commission must have confidence in your results in order to present those results to others.
- **Awareness of the data context:** All data have bias. Before, during and after analysis, it is essential to identify biases in the data and articulate clearly how these biases influence all steps of analysis and interpretation.

Table 9.1: File sizes. If you have limited storage, plan your analysis in stages.

DATA SET	FILE	ZIPPED	UNZIPPED
NCHS	US1969-1986.zip	2.7 GB	22.8 GB
	natalityConfBackup_PostgreSQL.sql	2.42 GB	25 GB
	US1969.zip	32.6 MB	381 MB
SEDAC	hauer_county_NH_pop_SSPs.xlsx	N/A	15.1 MB
SEER	SEER Data Dictionary.pdf		73 KB
	tx.1969_2020.19ages.adjusted.txt.gz	5.3 MB	35 MB
	tx.1969_2020.singleages.adjusted.txt.gz	18.8 MB	19 MB
	tx.1990_2020.19ages.adjusted.txt.gz	6.5 MB	6 MB
	tx.1990_2020.singleages.adjusted.txt.gz	20.9 MB	21 MB
	us.1969_2020.19ages.adjusted.txt.gz	66.7 MB	430 MB
	us.1969_2020.singleages.adjusted.txt.gz	238.8 MB	1.6 GB
	us.1990_2020.19ages.adjusted.txt.gz	76.7 MB	520 MB
	us.1990_2020.singleages.adjusted.txt.gz	246.3 MB	1.8 GB
TOTAL		5.7 GB	52 GB

- **Reproducible results:** You must enable the commission to have your results confirmed by an independent team. That is, enable the independent team to replicate your results by describing your data and methods in detail.

Table 9.2: Evaluation Criteria

CRITERION	% WEIGHT
1. Compelling presentation - Informative	10%
2. Compelling presentation - Understandable	10%
3. Analysis comprehension	20%
4. Sound technical methods	20%
5. Awareness of the data context	20%
6. Reproducible results	20%

Chapter 10

Analysis Activities

10.1 A single year of vital statistics

A problem most people can relate to is demography. Millions of people are born in the US every year. Recording birth events is necessary for legal matters such as obtaining a driver's license or other forms of government-issued identification. However, recording, keeping, and using this information has challenges that exemplify many aspects of data analysis, as this exercise will demonstrate.

DO NOT USE ANY FORM OF AI HERE

There is a reason for the request not to use AI for the analysis tasks, e.g. LLMs. **DO NOT DO IT.** If you do it, you will lose an essential component of training. During the workshop, we will discuss how to solve the problems in this section and more advanced problems with AI. We will explain the rationale for this request within the first hour of the in-person workshop.

Since 1969, all births recorded in the US are available as digital records from the National Center for Health Statistics (NCHS) from the Centers for Disease Control and Prevention (CDC). Only between 1969 and 1988 the date and time of birth is publicly available; starting on 1989, only the week of birth is recorded. Why is the date and time of birth no longer recorded? **Record your explanation as answer #26.**

The NCHS keeps records of place of birth, assistance during delivery (at home, with doctors, with midwives), level of education of the parents, place of residence, weight at birth, number of weeks of gestation, number of siblings, birth order, etc. In total, over 100 variables are recorded.

I have made available two files for you: a compressed file with birth records from 1969, and a data dictionary. The uncompressed data file is about 380 MB in size. The data is contained in a "flat file". This means that every line of text in this file is a continuous chain of characters. We must extract information from this type of files with a "dictionary" that tells us the beginning and ending columns of a given variable. Why was this format used? **Record your explanation as answer #27.**

Please answer the following questions:

1. How many live births occurred in Texas in 1969 from mothers residing in Texas? **Record your explanation as answer #28.**
 - Bonus question: How would you visualize births from each state with respect to every other state?
2. Show graphically how the level of education of the mother is related to the birth order (1st born, second child, third, etc.) **Record your explanation as answer #29.**
 - Bonus question: How would you visualize each variable with respect to every other variable?

It is possible that you might not know how to answer some of these questions on first contact with this problem.

A simple program that can help you explore this file is

```
1 f = open("US1969.dat", "r", encoding="cp1252")
2 counter = 0;
3 for x in f:
4     if x[0] == "9" and [... other conditions?]
5         counter = counter + 1
6 print(counter)
```

Note that in Python indexing is subtle. For example:

```
1 x = [10, 20, 30, 40, 50]
2 x[1:3] # This is [20, 30] (indices 1 and 2, not 3)
```

The instruction `encoding="cp1252"` is necessary in non-Windows systems due the presence of single-byte character encoding of the Latin alphabet, used by default in the legacy components of Microsoft Windows for English and many European languages including Spanish, French, and German. If you remove this instruction, the following error might show up in non-Windows operating systems: “utf-8’ codec can’t decode byte...”

A common approach to extract information from flat files is by importing it into Excel. The “*Text Import Wizard*” (typically shown as in Figure 1) would guide you through the process of identifying variables by column. However, this results in a problem. Describe it. **Record your explanation as answer #30.**

Text Import Wizard - Step 1 of 3

The Text Wizard has determined that your data is Fixed Width.
If this is correct, choose Next, or choose the data type that best describes your data.

Original data type

Choose the file type that best describes your data:

☐ Delimited - Characters such as commas or tabs separate each field.

☒ Fixed width - Fields are aligned in columns with spaces between each field.

Start import at row: 1 File origin: 437 : OEM United States

☐ My data has headers.

Preview of file D:\ResearchData\Lunar\US1969.dat.

1	9010	110100199990002630100163132223231012083320400000550555552	26043
2	9010	220102699990002630100163111111200709083320100000220222222	19023
3	9010	110100199990002630100163111111210810083320300000440444442	27041
4	9010	110100199990002630100163111111271414094430200000330333332	27043
5	9010	110100199990002630100163139223261314094430502000880887652	99112

< >

Cancel < Back Next > Finish

Figure 1: Excel's Import Wizard